# Chapter 5 ▪ Database Programming

In 1996, Sun released the first version of the JDBC API. This API lets programmers connect to a database to query or update it using the Structured Query Language (SQL). (SQL, usually pronounced “sequel,” is an industry standard for relational database access.) JDBC has since become one of the most commonly used APIs in the Java library.

JDBC has been updated several times. As this book is published, JDBC 4.3, the version included with Java 9, is the most current version.

In this chapter, I will explain the key ideas behind JDBC. I will introduce you to (or refresh your memory of) SQL, the industry-standard Structured Query Language for relational databases. The chapter has enough details and examples to let you start using JDBC for common programming situations.

---



**Note:** According to Oracle, JDBC is a trademarked term and not an acronym for Java Database Connectivity. It was named to be reminiscent of ODBC, a standard database API pioneered by Microsoft and since incorporated into the SQL standard.

---

## 5.1. The Design of JDBC

From the start, the developers of the Java technology were aware of the potential that Java showed for working with databases. In 1995, they began work on extending the standard Java library to deal with SQL access to databases. What they first hoped to do was to extend Java so that a program could talk to any random database using only “pure” Java. It didn’t take them long to realize that this is an impossible task: There are simply too many databases out there, using too many protocols. Moreover, although database vendors were all in favor of Java providing a standard network protocol for database access, they were only in favor of it if Java used *their* network protocol.

What all the database vendors and tool vendors *did* agree on was that it would be useful for Java to provide a pure Java API for SQL access along with a driver manager to allow third-party drivers to connect to specific databases. Database vendors could provide their own drivers to plug into the driver manager. There would then be a simple mechanism for registering third-party drivers with the driver manager.

This organization follows the very successful model of Microsoft’s ODBC which provided a C programming language interface for database access. Both JDBC and ODBC are based on the same idea: Programs written according to the API talk to the driver manager, which, in turn, uses a driver to talk to the actual database.

This means the JDBC API is all that most programmers will ever have to deal with.

### 5.1.1. JDBC Driver Types

The JDBC specification classifies drivers into the following *types*:

- A *type 1 driver* translates JDBC to ODBC and relies on an ODBC driver to communicate with the database. Early versions of Java included one such driver, the *JDBC/ODBC bridge*. However, the bridge requires de-

ployment and proper configuration of an ODBC driver. When JDBC was first released, the bridge was handy for testing, but it was never intended for production use. At this point, many better drivers are available, and the JDK no longer provides the JDBC/ODBC bridge.

- A *type 2 driver* is written partly in Java and partly in native code; it communicates with the client API of a database. To use such a driver, you must install some platform-specific code onto the client in addition to a Java library.
- A *type 3 driver* is a pure Java client library that uses a database-independent protocol to communicate database requests to a server component, which then translates the requests into a database-specific protocol. This simplifies deployment because the platform-specific code is located only on the server.
- A *type 4 driver* is a pure Java library that translates JDBC requests directly to a database-specific protocol.

---



**Note:** The JDBC specification is available at **https://jcp.org/aboutJava/communityprocess/mrel/jsr221/index3.html**.

---

Most database vendors supply either a type 3 or type 4 driver with their database. Furthermore, a number of third-party companies specialize in producing drivers with better standards conformance, support for more platforms, better performance, or, in some cases, simply better reliability than the drivers provided by the database vendors.

In summary, the ultimate goal of JDBC is to make possible the following:

- Programmers can write applications in the Java programming language to access any database using standard SQL statements (or even specialized extensions of SQL) while still following Java language conventions.
- Database vendors and database tool vendors can supply the low-level drivers. Thus, they can optimize their drivers for their specific products.

---



**Note:** If you are curious as to why Java just didn't adopt the ODBC model, the reason, as given at the JavaOne conference in 1996, was this:

- ODBC is hard to learn.
- ODBC has a few commands with lots of complex options. The preferred style in the Java programming language is to have simple and intuitive methods, but to have lots of them.
- ODBC relies on the use of `void*` pointers and other C features that are not natural in the Java programming language.
- An ODBC-based solution is inherently less safe and harder to deploy than a pure Java solution.

---

### 5.1.2. Typical Uses of JDBC

The traditional client/server model has a rich GUI on the client and a database on the server (see **Figure 5.1**). In this model, a JDBC driver is deployed on the client.

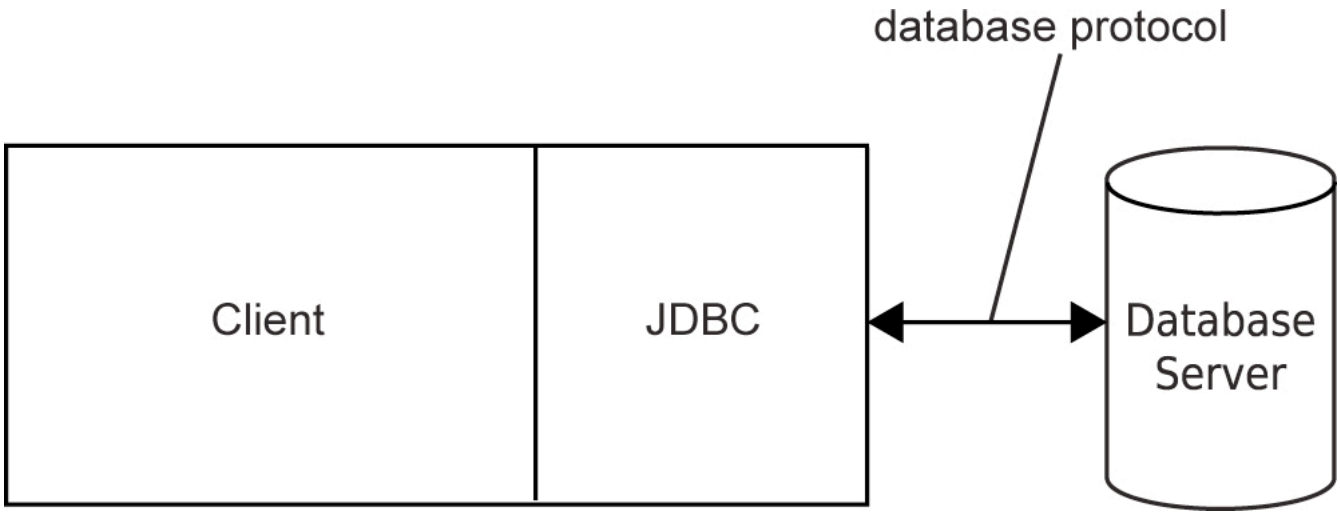



**Figure 5.1:** A traditional client/server application

However, nowadays it is far more common to have a three-tier model where the client application does not make database calls. Instead, it calls on a middleware layer on the server that in turn makes the database queries. The three-tier model has a couple of advantages. It separates *visual presentation* (on the client) from the *business logic* (in the middle tier) and the raw data (in the database). Therefore, it becomes possible to access the same data and the same business rules from multiple clients, such as a Java desktop application, a web browser, or a mobile app.

Communication between the client and the middle tier typically occurs through HTTP. JDBC manages the communication between the middle tier and the back-end database. **Figure 5.2** shows the basic architecture.

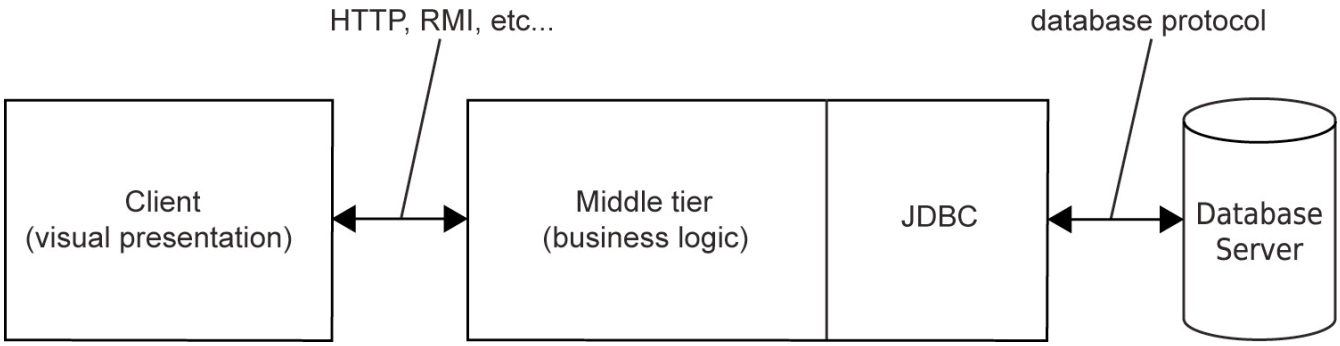



**Figure 5.2:** A three-tier application

## 5.2. The Structured Query Language

JDBC lets you communicate with databases using SQL, which is the command language for essentially all relational database servers. Desktop databases such as Microsoft Access have a GUI for queries and updates, but JDBC is focused on server-based databases such as Oracle, PostgreSQL, MySQL, Microsoft SQL Server, and so on.

The JDBC package can be thought of as nothing more than an API for communicating SQL statements to databases. We will briefly introduce SQL in this section. If you have never seen SQL before, you might not find this material sufficient. If so, turn to one of the many learning resources on the topic; I recommend *Learning SQL* by Alan Beaulieu (O'Reilly, 2009) or the online book *Learn SQL The Hard Way* at **https://sql.learncodethehardway.org**.

You can think of a database as a bunch of named tables with rows and columns. Each column has a *column name*. Each row contains a set of related data.

As an example database for this book, we use a set of database tables that describe a collection of classic computer science books.

**Table 5.1:** The `Authors` Table

| Author_Id | Name | Fname |
|---|---|---|
| ALEX | Alexander | Christopher |
| BROO | Brooks | Frederick P. |
| . . . | . . . | . . . |

**Table 5.2:** The `Books` Table

| Title | ISBN | Publisher_Id | Price |
|---|---|---|---|
| A Guide to the SQL Standard | 0-201-96426-0 | 0201 | 47.95 |
| A Pattern Language: Towns, Buildings, Construction | 0-19-501919-9 | 019 | 65.00 |
| . . . | . . . | . . . | . . . |

**Table 5.3:** The `BooksAuthors` Table

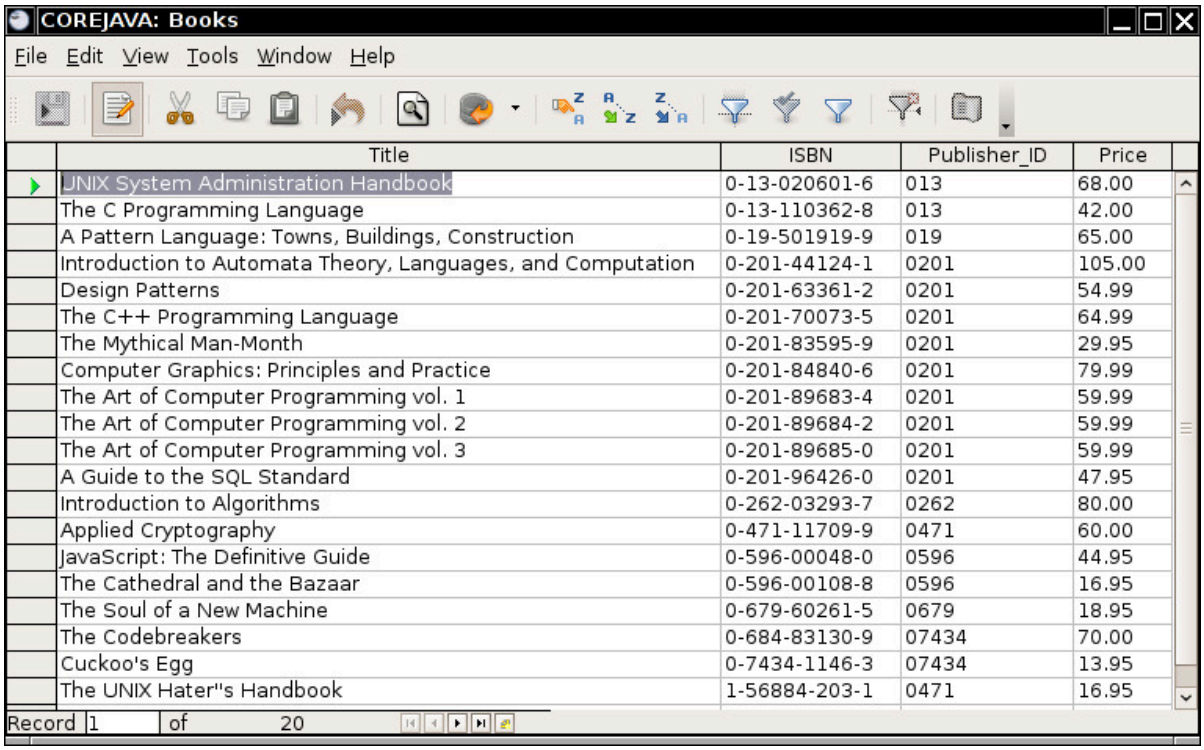
| ISBN | Author_Id | Seq_No |
|---|---|---|
| 0-201-96426-0 | DATE | 1 |
| 0-201-96426-0 | DARW | 2 |
| 0-19-501919-9 | ALEX | 1 |
| . . . | . . . | . . . |

**Table 5.4:** The `Publishers` Table

| Publisher_Id | Name | URL |
|---|---|---|
| 0201 | Addison-Wesley | www.aw-bc.com |
| 0407 | John Wiley & Sons | www.wiley.com |
| . . . | . . . | . . . |

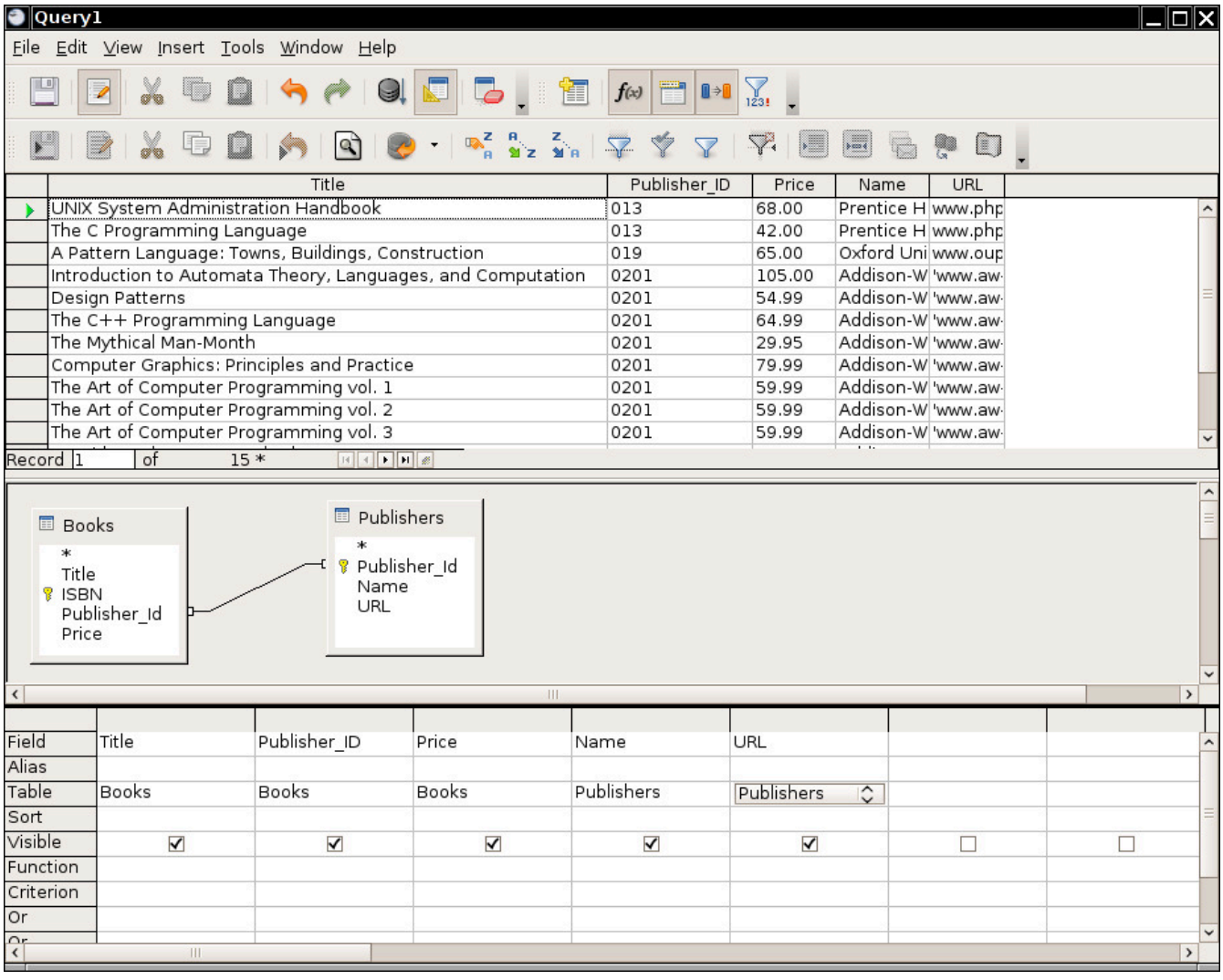
**Figure 5.3** shows a view of the `Books` table. **Figure 5.4** shows the result of *joining* this table with the `Publishers` table. The `Books` and the `Publishers` tables each contain an identifier for the publisher.

When we join both tables on the publisher code, we obtain a *query result* made up of values from the joined tables. Each row in the result contains the information about a book, together with the publisher name and web page URL. Note that the publisher names and URLs are duplicated across several rows because we have several books with the same publisher.




| Title | ISBN | Publisher_ID | Price |
|---|---|---|---|
| UNIX System Administration Handbook | 0-13-020601-6 | 013 | 68.00 |
| The C Programming Language | 0-13-110362-8 | 013 | 42.00 |
| A Pattern Language: Towns, Buildings, Construction | 0-19-501919-9 | 019 | 65.00 |
| Introduction to Automata Theory, Languages, and Computation | 0-201-44124-1 | 0201 | 105.00 |
| Design Patterns | 0-201-63361-2 | 0201 | 54.99 |
| The C++ Programming Language | 0-201-70073-5 | 0201 | 64.99 |
| The Mythical Man-Month | 0-201-83595-9 | 0201 | 29.95 |
| Computer Graphics: Principles and Practice | 0-201-84840-6 | 0201 | 79.99 |
| The Art of Computer Programming vol. 1 | 0-201-89683-4 | 0201 | 59.99 |
| The Art of Computer Programming vol. 2 | 0-201-89684-2 | 0201 | 59.99 |
| The Art of Computer Programming vol. 3 | 0-201-89685-0 | 0201 | 59.99 |
| A Guide to the SQL Standard | 0-201-96426-0 | 0201 | 47.95 |
| Introduction to Algorithms | 0-262-03293-7 | 0262 | 80.00 |
| Applied Cryptography | 0-471-11709-9 | 0471 | 60.00 |
| JavaScript: The Definitive Guide | 0-596-00048-0 | 0596 | 44.95 |
| The Cathedral and the Bazaar | 0-596-00108-8 | 0596 | 16.95 |
| The Soul of a New Machine | 0-679-60261-5 | 0679 | 18.95 |
| The Codebreakers | 0-684-83130-9 | 07434 | 70.00 |
| Cuckoo's Egg | 0-7434-1146-3 | 07434 | 13.95 |
| The UNIX Hater"s Handbook | 1-56884-203-1 | 0471 | 16.95 |

**Figure 5.3:** Sample table containing books




| Title | Publisher_ID | Price | Name | URL |
|---|---|---|---|---|
| UNIX System Administration Handbook | 013 | 68.00 | Prentice H | www.php |
| The C Programming Language | 013 | 42.00 | Prentice H | www.php |
| A Pattern Language: Towns, Buildings, Construction | 019 | 65.00 | Oxford Uni | www.oup |
| Introduction to Automata Theory, Languages, and Computation | 0201 | 105.00 | Addison-W | 'www.aw· |
| Design Patterns | 0201 | 54.99 | Addison-W | 'www.aw· |
| The C++ Programming Language | 0201 | 64.99 | Addison-W | 'www.aw· |
| The Mythical Man-Month | 0201 | 29.95 | Addison-W | 'www.aw· |
| Computer Graphics: Principles and Practice | 0201 | 79.99 | Addison-W | 'www.aw· |
| The Art of Computer Programming vol. 1 | 0201 | 59.99 | Addison-W | 'www.aw· |
| The Art of Computer Programming vol. 2 | 0201 | 59.99 | Addison-W | 'www.aw· |
| The Art of Computer Programming vol. 3 | 0201 | 59.99 | Addison-W | 'www.aw· |

**Figure 5.4:** Two tables joined together

The benefit of joining tables is avoiding unnecessary duplication of data in the database tables. For example, a naive database design might have had columns for the publisher name and URL right in the `Books` table. But then the database itself, and not just the query result, would have many duplicates of these entries. If a publisher's web address changed, *all* entries would need to be updated. Clearly, this is somewhat error-prone. In the relational model, we distribute data into multiple tables so that no information is unnecessarily duplicated. For example, each publisher's URL is contained only once in the publisher table. If the information needs to be combined, the tables are joined.

In the figures, you can see a graphical tool to inspect and link the tables. Many vendors have tools to express queries in a simple form by connecting column names and filling information into forms. Such tools are often called *query by example* (QBE) tools. In contrast, a query that uses SQL is written out in text, using SQL syntax, for example:

```
SELECT Books.Title, Books.Publisher_Id, Books.Price, Publishers.Name, Publishers.URL
FROM Books, Publishers
WHERE Books.Publisher_Id = Publishers.Publisher_Id
```

In the remainder of this section, you will learn how to write such queries. If you are already familiar with SQL, just skip this section.

By convention, SQL keywords are written in capital letters, although this is not necessary.

The `SELECT` statement is quite flexible. You can simply select all rows in the `Books` table with the following query:

```
SELECT * FROM Books
```

The `FROM` clause is required in every SQL `SELECT` statement. It tells the database which tables to examine to find the data.

You can choose the columns that you want:

```
SELECT ISBN, Price, Title
FROM Books
```

You can restrict the rows in the answer with the `WHERE` clause:

```
SELECT ISBN, Price, Title
FROM Books
WHERE Price <= 29.95
```

Be careful with the "equals" comparison. SQL uses `=` and `<>`, rather than `==` or `!=` as in the Java programming language, for equality testing.

---



**Note:** Some database vendors support the use of `!=` for inequality testing. This is not standard SQL, so I recommend against such use.

---

The `WHERE` clause can also use pattern matching by means of the `LIKE` operator. The wildcard characters are not the usual `*` and `?`, however. Use a `%` for zero or more characters and an underscore for a single character. For example,

```
SELECT ISBN, Price, Title
FROM Books
WHERE Title NOT LIKE '%n_x%'
```

excludes books with titles that contain words such as Unix or Linux.

Note that strings are enclosed in single quotes, not double quotes. A single quote inside a string is represented by a pair of single quotes. For example,

```
SELECT Title
FROM Books
WHERE Title LIKE '%''%'
```

reports all titles that contain a single quote.

You can select data from multiple tables:

```
SELECT * FROM Books, Publishers
```

Without a `WHERE` clause, this query is not very interesting. It lists *all combinations* of rows from both tables. In our case, where `Books` has 20 rows and `Publishers` has 8 rows, the result is a set of rows with 20 × 8 entries and lots of duplications. We really want to constrain the query to say that we are only interested in *matching* books with their publishers:

```
SELECT * FROM Books, Publishers
WHERE Books.Publisher_Id = Publishers.Publisher_Id
```

This query result has 20 rows, one for each book, because each book has one publisher in the `Publisher` table.

Whenever you have multiple tables in a query, the same column name can occur in two different places. That happened in our example. There is a column called `Publisher_Id` in both the `Books` and the `Publishers` tables. When an ambiguity would otherwise result, you must prefix each column name with the name of the table to which it belongs, such as `Books.Publisher_Id`.

You can use SQL to change the data inside a database as well. For example, suppose you want to reduce by $5.00 the current price of all books that have “C++” in their title:

```
UPDATE Books
SET Price = Price - 5.00
WHERE Title LIKE '%C++%'
```

Similarly, to delete all C++ books, use a `DELETE` query:

```
DELETE FROM Books
WHERE Title LIKE '%C++%'
```

SQL comes with built-in functions for taking averages, finding maximums and minimums in a column, and so on, which we do not discuss here.

Typically, to insert values into a table, you can use the `INSERT` statement:

```
INSERT INTO Books
VALUES ('A Guide to the SQL Standard', '0-201-96426-0', '0201', 47.95)
```

You need a separate `INSERT` statement for every row being inserted in the table.

Of course, before you can query, modify, and insert data, you must have a place to store data. Use the `CREATE TABLE` statement to make a new table. Specify the name and data type for each column. For example,

```
CREATE TABLE Books
(
   Title VARCHAR(60),
   ISBN CHAR(13),
   Publisher_Id VARCHAR(6),
   Price DECIMAL(10,2)
)
```

**Table 5.5** shows the most common SQL data types.

**Table 5.5:** Common SQL Data Types

| Data Types | Description |
|---|---|
| `INTEGER` or `INT` | Typically, a 32-bit integer |
| `SMALLINT` | Typically, a 16-bit integer |
| `NUMERIC(m,n)`, `DECIMAL(m,n)` or `DEC(m,n)` | Fixed-point decimal number with `m` total digits and `n` digits after the decimal point |
| `FLOAT(n)` | A floating-point number with `n` binary digits of precision |
| `REAL` | Typically, a 32-bit floating-point number |
| `DOUBLE` | Typically, a 64-bit floating-point number |
| `CHARACTER(n)` or `CHAR(n)` | Fixed-length string of length `n` |
| `VARCHAR(n)` | Variable-length strings of maximum length `n` |

| Data Types | Description |
|---|---|
| `BOOLEAN` | A Boolean value |
| `DATE` | Calendar date, implementation-dependent |
| `TIME` | Time of day, implementation-dependent |
| `TIMESTAMP` | Date and time of day, implementation-dependent |
| `BLOB` | A binary large object |
| `CLOB` | A character large object |

In this book, we do not discuss the additional clauses, such as keys and constraints, that you can use with the `CREATE TABLE` statement.

## 5.3. JDBC Configuration

Of course, you need a database program for which a JDBC driver is available. There are many excellent choices, such as IBM DB2, Microsoft SQL Server, MySQL, Oracle, and PostgreSQL.

You must also create a database for your experimental use. We assume you name it `COREJAVA`. Create a new database, or have your database administrator create one with the appropriate permissions. You need to be able to create, update, and drop tables in the database.

If this is your first experience with databases, I recommend that you use the Apache Derby database, which is available from **https://db.apache.org/derby**.

Alternatively, if you are familiar with Docker, it is easy to run PostgreSQL in a docker image:

```
docker pull postgres
docker run -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=COREJAVA -p 5432:5432 -d postgres
```

You need to gather a number of items before you can write your first database program. The following sections cover these items.

### 5.3.1. Database URLs

When connecting to a database, you must use various database-specific parameters such as host names, port numbers, and database names.

JDBC uses a syntax similar to that of ordinary URLs to describe data sources. Here are examples of the syntax:

```
jdbc:derby://localhost:1527/COREJAVA;create=true
jdbc:postgresql:COREJAVA
```

These JDBC URLs specify a Derby database and a PostgreSQL database named `COREJAVA`.

The general syntax is

```
jdbc:subprotocol:other stuff
```

where a subprotocol selects the specific driver for connecting to the database.

The format for the other stuff depends on the subprotocol used. You will need to look up your vendor's documentation for the specific format.

### 5.3.2. Driver JAR Files

You need to obtain the JAR file in which the driver for your database is located. If you use Derby, you need the file `derbyclient.jar`. With another database, you need to locate the appropriate driver. For example, the PostgreSQL drivers are available at **https://jdbc.postgresql.org**.

Include the driver JAR file on the class path when running a program that accesses the database. (You don't need the JAR file for compiling.)

When you launch programs from the command line, simply use the command

```
java -classpath driverJarPath:. ProgramName
```

On Windows, use a semicolon to separate the current directory (denoted by the `.` character) from the driver JAR path.

### 5.3.3. Starting the Database

The database server needs to be started before you can connect to it. The details depend on your database.

With the Derby database, follow these steps:

1. Open a command shell and change to a directory that will hold the database files.
2. Locate the file `derbyrun.jar` in the Derby installation directory. We will denote the directory containing `lib/derbyrun.jar` with `derby`.
3. Run the command

   ```
   java -jar derby/lib/derbyrun.jar server start
   ```

4. Double-check that the database is working correctly. Create a file `ij.properties` that contains these lines:

   ```
   ij.driver=org.apache.derby.jdbc.ClientDriver
   ij.protocol=jdbc:derby://localhost:1527/
   ij.database=COREJAVA;create=true
   ```

   From another command shell, run Derby's interactive scripting tool (called `ij`) by executing

   ```
   java -jar derby/lib/derbyrun.jar ij -p ij.properties
   ```

Now you can issue SQL commands such as

```
CREATE TABLE Greetings (Message VARCHAR(20));
INSERT INTO Greetings VALUES ('Hello, World!');
SELECT * FROM Greetings;
DROP TABLE Greetings;
```

Note that each command must be terminated by a semicolon. To exit, type

```
EXIT;
```

5. When you are done using the database, stop the server with the command

```
java -jar derby/lib/derbyrun.jar server shutdown
```

If you use another database, you need to consult the documentation to find out how to start and stop your database server, and how to connect to it and issue SQL commands.

### 5.3.4. Connecting to the Database

In your Java program, open a database connection like this:

```
String url = "jdbc:postgresql:COREJAVA";
String username = "dbuser";
String password = "secret";
Connection conn = DriverManager.getConnection(url, username, password);
```

The driver manager iterates through the registered drivers to find a driver that can use the subprotocol specified in the database URL.

The `getConnection` method returns a `Connection` object. In the following sections, you will see how to use the `Connection` object to execute SQL statements.

To connect to the database, you will need a username and password for your database.

---



**Note:** By default, Derby lets you connect with any username, and it does not check passwords. A separate set of tables is generated for each user. The default username is `app`.

---

The test program in **Listing 5.1** puts these steps to work. It loads connection parameters from a file named `database.properties` and connects to the database. The `database.properties` file supplied with the sample code contains connection information for the Derby database. If you use a different database, put your database-specific connection information into that file. Here is an example for connecting to a PostgreSQL database:

```
jdbc.drivers=org.postgresql.Driver
jdbc.url=jdbc:postgresql:COREJAVA
jdbc.username=dbuser
jdbc.password=secret
```

After connecting to the database, the test program executes the following SQL statements:

```
CREATE TABLE Greetings (Message VARCHAR(20))
INSERT INTO Greetings VALUES ('Hello, World!')
SELECT * FROM Greetings
```

The result of the `SELECT` statement is printed, and you should see an output of

```
Hello, World!
```

Then the table is removed by executing the statement

```
DROP TABLE Greetings
```

To run this test, start your database as described previously, and launch the program as

```
java -classpath driverJarPath:. test.TestDB
```



**Tip:** One way to debug JDBC-related problems is to enable JDBC tracing. Call the `DriverManager.setLogWriter` method to send trace messages to a `PrintWriter`. The trace output contains a detailed listing of the JDBC activity. Most JDBC driver implementations provide additional mechanisms for tracing. For example, with Derby, you can add a `traceFile` option to the JDBC URL: `jdbc:derby://localhost:1527/COREJAVA;create=true;traceFile=trace.out`.



**Note:** Since JDBC 4, driver JAR files contain a file `META-INF/services/java.sql.Driver` which contains the name of the driver class.

In the unlikely case that you use an older driver JAR, you need to find out the name of the JDBC driver class used by your vendor. Then set the `jdbc.drivers` property on the command line:

```
java -Djdbc.drivers=driverClassName ProgramName
```

**Listing 5.1 v2ch05/test/TestDB.java**

```
1  package v2ch05.test;
2
3  import module java.base;
4  import module java.sql;
5  import v2ch05.exec.ExecSQL;
6
7  /**
8   * This program tests that the database and the JDBC driver are correctly configured.
9   */
10 class TestDB {
11     void main() throws Exception {
12         try {
13             runTest();
```

```
        }
        catch (SQLException e) {
            for (Throwable t : e)
                t.printStackTrace();
        }
    }

    /**
     * Runs a test by creating a table, adding a value, showing the table contents, and
     * removing the table.
     */
    void runTest() throws SQLException, IOException {
        try (Connection conn = ExecSQL.getConnection()) {
            Statement stat = conn.createStatement();
            stat.executeUpdate("CREATE TABLE Greetings (Message VARCHAR(20))");
            stat.executeUpdate("INSERT INTO Greetings VALUES ('Hello, World!')");

            try (ResultSet result = stat.executeQuery("SELECT * FROM Greetings")) {
                if (result.next()) System.out.println(result.getString(1));
            }
            stat.executeUpdate("DROP TABLE Greetings");
        }
    }
}
```

`java.sql.DriverManager` 1.1

- `static Connection getConnection(String url, String user, String password)` establishes a connection to the given database and returns a `Connection` object.

## 5.4. Working with JDBC Statements

In the following sections, you will see how to use the JDBC `Statement` to execute SQL statements, obtain results, and deal with errors. Then we show you a simple program for populating a database.

### 5.4.1. Executing SQL Statements

To execute a SQL statement, you first create a `Statement` object. To create statement objects, use the `Connection` object that you obtained from the call to `DriverManager.getConnection`.

```
Statement stat = conn.createStatement();
```

Next, place the statement that you want to execute into a string, for example:

```
String command = """
UPDATE Books
SET Price = Price - 5.00
WHERE Title NOT LIKE '%Introduction%'
""";
```

Then, call the `executeUpdate` method of the `Statement` interface:

```
stat.executeUpdate(command);
```

The `executeUpdate` method returns a count of the rows that were affected by the SQL statement, or zero for statements that do not return a row count. For example, the call to `executeUpdate` in the preceding example returns the number of rows where the price was lowered by $5.00.

The `executeUpdate` method can execute actions such as `INSERT`, `UPDATE`, and `DELETE`, as well as data definition statements such as `CREATE TABLE` and `DROP TABLE`. However, you need to use the `executeQuery` method to execute `SELECT` queries. There is also a catch-all `execute` statement to execute arbitrary SQL statements. It's commonly used only for queries that a user supplies interactively.

When you execute a query, you are interested in the result. The `executeQuery` method returns an object of type `ResultSet` that you can use to walk through the result one row at a time.

```
ResultSet rs = stat.executeQuery("SELECT * FROM Books");
```

The basic loop for analyzing a result set looks like this:

```
while (rs.next()) {
     look at a row of the result set
}
```

---



**Caution:** The iteration protocol of the `ResultSet` interface is subtly different from that of the `java.util.Iterator` interface. Here, the iterator is initialized to a position *before* the first row. You must call the `next` method once to move the iterator to the first row. Also, there is no `hasNext` method; keep calling `next` until it returns `false`.

---

The order of the rows in a result set is completely arbitrary. Unless you specifically ordered the result with an `ORDER BY` clause, you should not attach any significance to the row order.

When inspecting an individual row, you will want to know the contents of the fields. A large number of accessor methods give you this information.

```
String isbn = rs.getString(1);
double price = rs.getDouble("Price");
```

There are accessors for various *types*, such as `getString` and `getDouble`. Each accessor has two forms, one taking a numeric argument, and the other, a string argument. When you supply a numeric argument, you refer to the column with that number. For example, `rs.getString(1)` returns the value of the first column in the current row.

---



**Caution:** Unlike array indexes, database column numbers start at `1`.

---

When you supply a string argument, you refer to the column in the result set with that name. For example, `rs.getDouble("Price")` returns the value of the column with label `Price`. Using the numeric argument is a bit more efficient, but string arguments make the code easier to read and maintain.

Each `get` method makes reasonable type conversions when the type of the method doesn't match the type of the column. For example, the call `rs.getString("Price")` converts the floating-point value of the `Price` column to a string.

*java.sql.Connection* **1.1**

- `Statement createStatement()`
  creates a `Statement` object that can be used to execute SQL queries and updates without parameters.
- `void close()`
  immediately closes the current connection and the JDBC resources that it created.

*java.sql.Statement* **1.1**

- `ResultSet executeQuery(String sqlQuery)`
  executes the SQL statement given in the string and returns a `ResultSet` object to view the query result.
- `int executeUpdate(String sqlStatement)`
- `long executeLargeUpdate(String sqlStatement)` **8**
  execute the SQL `INSERT`, `UPDATE`, or `DELETE` statement specified by the string. Also execute Data Definition Language (DDL) statements such as `CREATE TABLE`. Return the number of rows affected, or `0` for a statement without an update count.
- `boolean execute(String sqlStatement)`
  executes the SQL statement specified by the string. Multiple result sets and update counts may be produced. Returns `true` if the first result is a result set, `false` if the result is an update count. Call `getResultSet` or `getUpdateCount` to retrieve the first result. See for details on processing multiple results.
- `ResultSet getResultSet()`
  returns the result set of the preceding query statement, or `null` if the preceding statement did not have a result set. Call this method only once per executed statement.
- `int getUpdateCount()`
- `long getLargeUpdateCount()` **8**
  return the number of rows affected by the preceding update statement, or `-1` if the preceding statement was a statement without an update count. Call this method only once per executed statement.
- `void close()`
  closes this statement object and its associated result set.
- `boolean isClosed()` **6**
  returns `true` if this statement is closed.
- `void closeOnCompletion()` **7**
  causes this statement to be closed once all of its result sets have been closed.

*java.sql.ResultSet* **1.1**

- `boolean next()`
  makes the current row in the result set move forward by one. Returns `false` after the last row. Note that you must call this method to advance to the first row.
- `Xxx getXxx(int columnNumber)`
- `Xxx getXxx(String columnLabel)`
  (Xxx is a type such as `int`, `double`, `String`, `Date`, etc.)

- `<T> T getObject(int columnIndex, Class<T> type)` 7
- `<T> T getObject(String columnLabel, Class<T> type)` 7
- `void updateObject(int columnIndex, Object x, SQLType targetSqlType)` 8
- `void updateObject(String columnLabel, Object x, SQLType targetSqlType)` 8
  return or update the value of the column with the given column index or label, converted to the specified type. The column label is the label specified in the SQL `AS` clause or the column name if `AS` is not used.
- `int findColumn(String columnName)`
  gives the column index associated with a column name.
- `void close()`
  immediately closes the current result set.
- `boolean isClosed()` 6
  returns `true` if this statement is closed.

### 5.4.2. Managing Connections, Statements, and Result Sets

Every `Connection` object can create one or more `Statement` objects. You can use the same `Statement` object for multiple unrelated commands and queries. However, a statement has *at most one* open result set. If you issue multiple queries whose results you analyze concurrently, you need multiple `Statement` objects.

Be forewarned, though, that there is a limit to the number of statements per connection. Use the `getMaxStatements` method of the `DatabaseMetaData` interface to find out the number of concurrently open statements that your JDBC driver supports.

In practice, you should probably not fuss with multiple concurrent result sets. If the result sets are related, you should be able to issue a combined query and analyze a single result. It is much more efficient to let the database combine queries than it is for a Java program to iterate through multiple result sets.

Be sure to finish processing of any result set before you issue a new query or update on a `Statement` object. The result sets of prior queries are automatically closed.

When you are done using a `ResultSet`, `Statement`, or `Connection`, it is a good idea to call the `close` method immediately. These objects use large data structures that draw on the finite resources of the database server.

The `close` method of a `Statement` object closes any associated result sets. Similarly, the `close` method of the `Connection` class closes all statements of the connection.

Conversely, you can call the `closeOnCompletion` method on a `Statement`, and it will close automatically as soon as all its result sets have closed.

If your connections are short-lived, don't worry about closing statements and result sets. To make sure a connection object cannot possibly remain open, use a `try`-with-resources statement:

```
try (Connection conn = . . .) {
    Statement stat = conn.createStatement();
    ResultSet result = stat.executeQuery(queryString);
    process query result
}
```

### 5.4.3. Analyzing SQL Exceptions

Each `SQLException` has a chain of `SQLException` objects that are retrieved with the `getNextException` method. This exception chain is in addition to the "cause" chain of `Throwable` objects that every exception has. (See **Chapter 7 of Volume I** for details about Java exceptions.) One would need two nested loops to fully enumerate all these exceptions. Fortunately, the `SQLException` class has been enhanced to implement the `Iterable<Throwable>` interface. The `iterator()` method yields an `Iterator<Throwable>` that iterates through both chains: It starts by going through the cause chain of the first `SQLException`, then moves on to the next `SQLException`, and so on. You can simply use an enhanced `for` loop:

```
for (Throwable t : sqlException) {
    do something with t
}
```

You can call `getSQLState` and `getErrorCode` on a `SQLException` to analyze it further. The first method yields a string that is standardized by either X/Open or SQL:2003. (Call the `getSQLStateType` method of the `DatabaseMetaData` interface to find out which standard is used by your driver.) The error code is vendor-specific.

The SQL exceptions are organized into an inheritance tree (shown in **Figure 5.5**). This allows you to catch specific error types in a vendor-independent way.

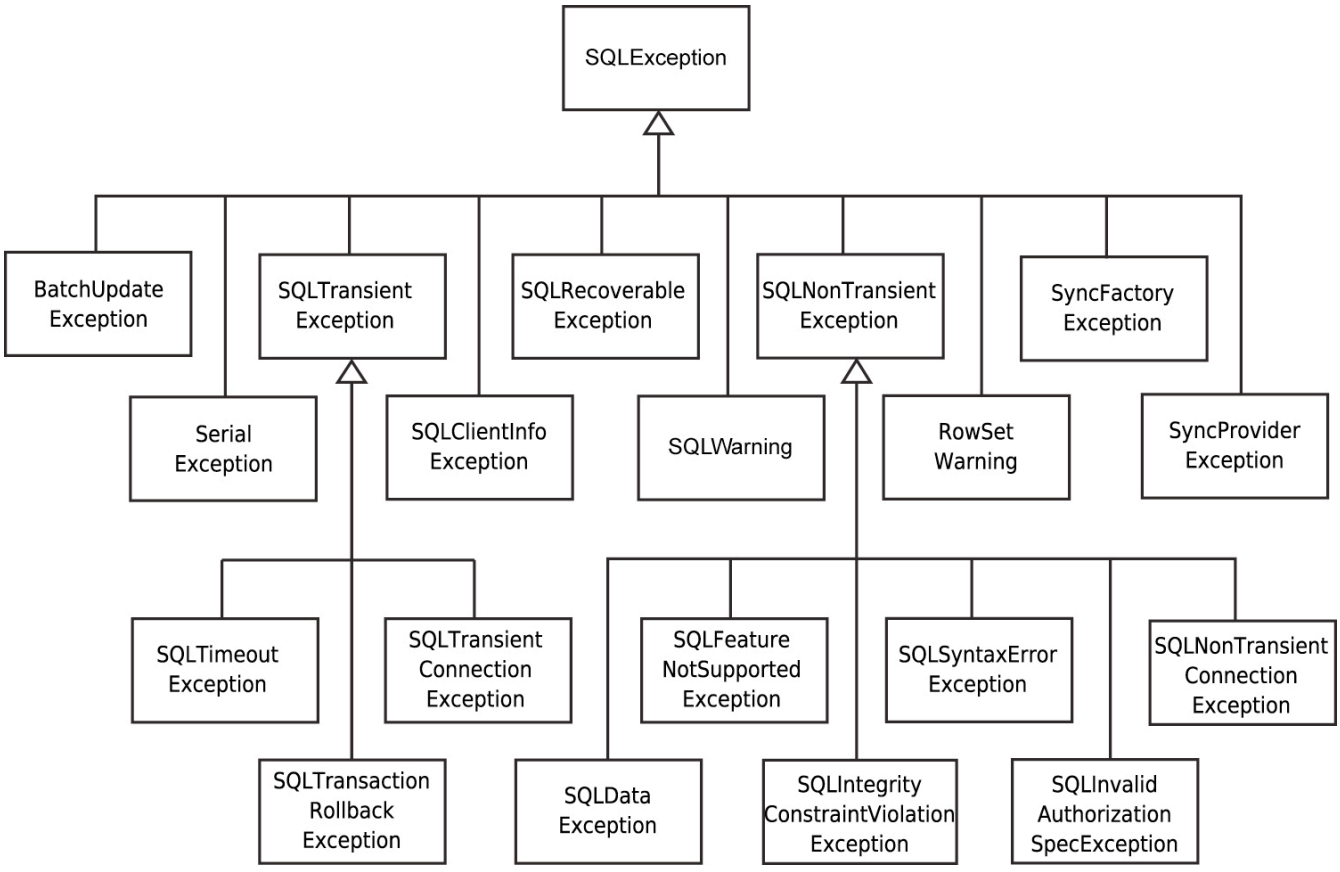



**Figure 5.5:** SQL exception types

In addition, the database driver can report nonfatal conditions as warnings. You can retrieve warnings from connections, statements, and result sets. The `SQLWarning` class is a subclass of `SQLException` (even though a `SQLWarning` is not thrown as an exception). Call `getSQLState` and `getErrorCode` to get further information about the warnings. Similar to SQL exceptions, warnings are chained. To retrieve all warnings, use this loop:

```
SQLWarning w = stat.getWarning();
while (w != null) {
    do something with w
    w = w.nextWarning();
}
```

The `DataTruncation` subclass of `SQLWarning` is used when data being read from the database are unexpectedly truncated. If data truncation happens in an update statement, a `DataTruncation` is thrown as an exception.

`java.sql.SQLException` 1.1

- `SQLException getNextException()`
  gets the next SQL exception chained to this one, or `null` at the end of the chain.
- `Iterator<Throwable> iterator()` 6
  gets an iterator that yields the chained SQL exceptions and their causes.
- `String getSQLState()`
  gets the "SQL state"—a standardized error code.
- `int getErrorCode()`
  gets the vendor-specific error code.

`java.sql.SQLWarning` 1.1

- `SQLWarning getNextWarning()`
  returns the next warning chained to this one, or `null` at the end of the chain.

*java.sql.Connection* 1.1 *java.sql.Statement* 1.1 *java.sql.ResultSet* 1.1

- `SQLWarning getWarnings()`
  returns the first of the pending warnings, or `null` if no warnings are pending.

`java.sql.DataTruncation` 1.1

- `boolean getParameter()`
  returns `true` if the data truncation applies to a parameter, `false` if it applies to a column.
- `int getIndex()`
  returns the index of the truncated parameter or column.
- `int getDataSize()`
  returns the number of bytes that should have been transferred, or `-1` if the value is unknown.
- `int getTransferSize()`
  returns the number of bytes that were actually transferred, or `-1` if the value is unknown.

### 5.4.4. Populating a Database

We are now ready to write our first real JDBC program. Sure, it would be nice to try some of the fancy queries discussed earlier, but we have a problem: Right now, there are no data in the database. We need to populate the database, and a simple way of doing that is with a set of SQL instructions to create tables and insert data

into them. Most database programs can process a set of SQL instructions from a text file, but there are pesky differences about statement terminators and other syntactical issues.

For that reason, we will use JDBC in a simple program that reads a file with SQL instructions, one instruction per line, and executes them.

Specifically, the program reads data from a text file in a format such as

```
CREATE TABLE Publishers (Publisher_Id VARCHAR(6), Name VARCHAR(30), URL VARCHAR(80));
INSERT INTO Publishers VALUES ('0201', 'Addison-Wesley', 'www.aw-bc.com');
INSERT INTO Publishers VALUES ('0471', 'John Wiley & Sons', 'www.wiley.com');
. . .
```

**Listing 5.2** contains the code for the program that reads a file with SQL statements and executes them. You don't have to read through the code; simply use the program so you can populate your database and run the examples in the remainder of this chapter.

Make sure that your database server is running, and run the program as follows:

```
java -classpath driverJarPath:. v2ch05.exec.ExecSQL v2ch05/Books.sql
java -classpath driverJarPath:. v2ch05.exec.ExecSQL v2ch05/Authors.sql
java -classpath driverJarPath:. v2ch05.exec.ExecSQL v2ch05/Publishers.sql
java -classpath driverJarPath:. v2ch05.exec.ExecSQL v2ch05/BooksAuthors.sql
```

Before running the program, check that the file `database.properties` is set up properly for your environment (see **Section 5.3.4**).

---



**Note:** Your database may also have a utility to process SQL files. For example, with Derby, you can run

```
java -jar derby/lib/derbyrun.jar ij -p ij.properties v2ch05/Books.sql
```

(The `ij.properties` file is described in **Section 5.3.3**.)

In the data format for the `ExecSQL` command, we allow an optional semicolon at the end of each line because most database utilities expect this format.

---

The following steps briefly describe the `ExecSQL` program:

1. Connect to the database. The `getConnection` method reads the properties in the file `database.properties` and adds the `jdbc.drivers` property to the system properties. The driver manager uses the `jdbc.drivers` property to load the appropriate database driver. The `getConnection` method uses the `jdbc.url`, `jdbc.username`, and `jdbc.password` properties to open the database connection.
2. Open the file with the SQL statements. If no file name was supplied, prompt the user to enter the statements on the console.
3. Execute each statement with the generic `execute` method. If it returns `true`, the statement had a result set. The four SQL files provided for the book database all end in a `SELECT *` statement so that you can see that the data were successfully inserted.

4. If there was a result set, print out the result. Since this is a generic result set, we need to use metadata to find out how many columns the result has. For more information, see **Section 5.8**.
5. If there is any SQL exception, print the exception and any chained exceptions that may be contained in it.
6. Close the connection to the database.

**Listing 5.2** shows the code for the program.

**Listing 5.2 v2ch05/exec/ExecSQL.java**

```
package v2ch05.exec;

import module java.base;
import module java.sql;

/**
 * Executes all SQL statements in a file or the console.
 */
public class ExecSQL {
    void main(String[] args) throws Exception {
        try (Scanner in = args.length == 0 ? new Scanner(System.in)
                : new Scanner(Path.of(args[0]));
            Connection conn = getConnection()) {

            Statement stat = conn.createStatement();
            while (true) {
                if (args.length == 0) IO.println("Enter command or EXIT to exit:");

                if (!in.hasNextLine()) return;

                String line = in.nextLine().strip();
                if (line.equalsIgnoreCase("EXIT")) return;
                if (line.endsWith(";")) // remove trailing semicolon
                    line = line.substring(0, line.length() - 1);
                try {
                    boolean isResult = stat.execute(line);
                    if (isResult) {
                        try (ResultSet rs = stat.getResultSet()) {
                            showResultSet(rs);
                        }
                    } else {
                        int updateCount = stat.getUpdateCount();
                        IO.println(updateCount + " rows updated");
                    }
                }
                catch (SQLException e) {
                    for (Throwable t : e)
                        t.printStackTrace();
                }
            }
        }
        catch (SQLException e) {
            for (Throwable t : e)
                t.printStackTrace();
        }
    }

```

```
48      /**
49       * Gets a connection from the properties specified in the file database.properties
50       * @return the database connection
51       */
52      public static Connection getConnection() throws SQLException, IOException {
53          var props = new Properties();
54          try (Reader in = Files.newBufferedReader(
55                  Path.of("v2ch05", "database.properties"))) {
56              props.load(in);
57          }
58          String drivers = props.getProperty("jdbc.drivers");
59          if (drivers != null) System.setProperty("jdbc.drivers", drivers);
60
61          String url = props.getProperty("jdbc.url");
62          String username = props.getProperty("jdbc.username");
63          String password = props.getProperty("jdbc.password");
64
65          return DriverManager.getConnection(url, username, password);
66      }
67
68      /**
69       * Prints a result set.
70       * @param result the result set to be printed
71       */
72      void showResultSet(ResultSet result) throws SQLException {
73          ResultSetMetaData metaData = result.getMetaData();
74          int columnCount = metaData.getColumnCount();
75
76          for (int i = 1; i <= columnCount; i++) {
77              if (i > 1) IO.print(", ");
78              IO.print(metaData.getColumnLabel(i));
79          }
80          IO.println();
81
82          while (result.next()) {
83              for (int i = 1; i <= columnCount; i++) {
84                  if (i > 1) IO.print(", ");
85                  IO.print(result.getString(i));
86              }
87              IO.println();
88          }
89      }
90  }
```

## 5.5. Query Execution

In this section, we write a program that executes queries against the `COREJAVA` database. For this program to work, you must have a `COREJAVA` database populated with tables as described in the preceding section.

When querying the database, you can select the author and the publisher or leave either of them as `Any`.

You can also change the data in the database. Select a publisher and type an amount. All prices of that publisher are adjusted by the amount you entered, and the program displays how many rows were changed. After a price change, you might want to run a query to verify the new prices.

### 5.5.1. Prepared Statements

In this program, we use one new feature, *prepared statements*. Consider the SQL query for all books by a particular publisher, regardless of the author:

```
SELECT Books.Price, Books.Title
FROM Books, Publishers
WHERE Books.Publisher_Id = Publishers.Publisher_Id
AND Publishers.Name = the name from the list box
```

Instead of building a separate query statement every time the user launches such a query, we can *prepare* a query with a host variable and use it many times, each time filling in a different string for the variable. That technique improves performance. Whenever the database executes a query, it first computes a strategy of how to do it efficiently. By preparing the query and reusing it, you ensure that the planning step is done only once.

Each host variable in a prepared query is indicated with a `?`. If there is more than one variable, you must keep track of the positions of the `?` when setting the values. For example, our prepared query becomes

```
String publisherQuery = """
SELECT Books.Price, Books.Title
FROM Books, Publishers
WHERE Books.Publisher_Id = Publishers.Publisher_Id AND Publishers.Name = ?
""";
PreparedStatement pstat = conn.prepareStatement(publisherQuery);
```

Before executing the prepared statement, you must bind the host variables to actual values with a `set` method. As with the `get` methods of the `ResultSet` interface, there are different `set` methods for the various types. Here, we want to set a string to a publisher name:

```
pstat.setString(1, publisher);
```

The first argument is the position number of the host variable that we want to set. The position 1 denotes the first `?`. The second argument is the value that we want to assign to the host variable.

If you reuse a prepared query that you have already executed, all host variables stay bound unless you change them with a `set` method or call the `clearParameters` method. That means you only need to call a `setXxx` method on those host variables that change from one query to the next.

Once all variables have been bound to values, you can execute the prepared statement:

```
ResultSet rs = pstat.executeQuery();
```

---



**Tip:** Building a query manually, by concatenating strings, is tedious and potentially dangerous. You have to worry about special characters such as quotes—and, if your query involves user input, you have to guard against injection attacks. Therefore, use prepared statements whenever your query involves variables.

---

The price update feature is implemented as an `UPDATE` statement. Note that we call `executeUpdate`, not `executeQuery`, because the `UPDATE` statement does not return a result set. The return value of `executeUpdate` is the count of changed rows.

```
int r = stat.executeUpdate();
IO.println(r + " rows updated");
```

---



**Note:** A `PreparedStatement` object becomes invalid after the associated `Connection` object is closed. However, many databases automatically *cache* prepared statements. If the same query is prepared twice, the database simply reuses the query strategy. Therefore, don't worry about the overhead of calling `prepareStatement`.

---

The following list briefly describes the structure of the example program:

- The author and publisher array lists are populated by running two queries that return all author and publisher names in the database.
- The queries involving authors are complex. A book can have multiple authors, so the `BooksAuthors` table stores the correspondence between authors and books. For example, the book with ISBN 0-201-96426-0 has two authors with codes `DATE` and `DARW`. The `BooksAuthors` table has the rows

  ```
  0-201-96426-0, DATE, 1
  0-201-96426-0, DARW, 2
  ```

  to indicate this fact. The third column lists the order of the authors. (We can't just use the position of the rows in the table. There is no fixed row ordering in a relational table.) Thus, the query has to join the `Books`, `BooksAuthors`, and `Authors` tables to compare the author name with the one selected by the user.

  ```
  SELECT Books.Price, Books.Title FROM Books, BooksAuthors, Authors, Publishers
  WHERE Authors.Author_Id = BooksAuthors.Author_Id AND BooksAuthors.ISBN = Books.ISBN
  AND Books.Publisher_Id = Publishers.Publisher_Id AND Authors.Name = ?
  AND Publishers.Name = ?
  ```

---



**Tip:** Some Java programmers avoid complex SQL statements such as this one. A surprisingly common, but very inefficient, workaround is to write lots of Java code that iterates through multiple result sets. But the database is *a lot* better at executing query code than a Java program can be—that's the core competency of a database. A rule of thumb: If you can do it in SQL, don't do it in Java.

---

- The `changePrices` method executes an `UPDATE` statement. Note that the `WHERE` clause of the `UPDATE` statement needs the publisher *code* and we only know the publisher *name*. This problem is solved with a nested subquery:

  ```
  UPDATE Books
  SET Price = Price + ?
  WHERE Books.Publisher_Id = (SELECT Publisher_Id FROM Publishers WHERE Name = ?)
  ```

Listing 5.3 is the complete program code.

**Listing 5.3 v2ch05/query/QueryDemo.java**

```
package v2ch05.query;

import module java.base;
import module java.sql;
import v2ch05.exec.ExecSQL;

/**
 * This program demonstrates several complex database queries.
 */
class QueryDemo {
    String allQuery = "SELECT Books.Price, Books.Title FROM Books";

    String authorPublisherQuery = """
SELECT Books.Price, Books.Title
FROM Books, BooksAuthors, Authors, Publishers
WHERE Authors.Author_Id = BooksAuthors.Author_Id
    AND BooksAuthors.ISBN = Books.ISBN
    AND Books.Publisher_Id = Publishers.Publisher_Id
    AND Authors.Name = ?
    AND Publishers.Name = ?
""";

    String authorQuery = """
SELECT Books.Price, Books.Title FROM Books, BooksAuthors, Authors
WHERE Authors.Author_Id = BooksAuthors.Author_Id"
    AND BooksAuthors.ISBN = Books.ISBN"
    AND Authors.Name = ?
""";

    String publisherQuery = """
SELECT Books.Price, Books.Title FROM Books, Publishers
WHERE Books.Publisher_Id = Publishers.Publisher_Id
    AND Publishers.Name = ?
""";

    String priceUpdate = """
UPDATE Books SET Price = Price + ? "
WHERE Books.Publisher_Id =
    (SELECT Publisher_Id FROM Publishers WHERE Name = ?)
""";

    ArrayList<String> authors = new ArrayList<>();
    ArrayList<String> publishers = new ArrayList<>();

    void main() throws Exception {
        try (Connection conn = ExecSQL.getConnection()) {
            authors.add("Any");
            publishers.add("Any");
            try (Statement stat = conn.createStatement()) {
                // Fill the authors array list
                String query = "SELECT Name FROM Authors";
                try (ResultSet rs = stat.executeQuery(query)) {
                    while (rs.next())
```

```
                        authors.add(rs.getString(1));
                }

                // Fill the publishers array list
                query = "SELECT Name FROM Publishers";
                try (ResultSet rs = stat.executeQuery(query)) {
                    while (rs.next())
                        publishers.add(rs.getString(1));
                }
            }
            boolean done = false;
            while (!done) {
                String input = IO.readln("Q)uery C)hange prices E)xit: ").toUpperCase();
                if (input.equals("Q"))
                    executeQuery(conn);
                else if (input.equals("C"))
                    changePrices(conn);
                else
                    done = true;
            }
        }
        catch (SQLException e) {
            for (Throwable t : e)
                IO.println(t.getMessage());
        }
    }

    /**
     * Executes the selected query.
     * @param conn the database connection
     */
    void executeQuery(Connection conn) throws SQLException {
        String author = select("Authors:", authors);
        String publisher = select("Publishers:", publishers);
        PreparedStatement stat;
        if (!author.equals("Any") && !publisher.equals("Any")) {
            stat = conn.prepareStatement(authorPublisherQuery);
            stat.setString(1, author);
            stat.setString(2, publisher);
        }
        else if (!author.equals("Any") && publisher.equals("Any")) {
            stat = conn.prepareStatement(authorQuery);
            stat.setString(1, author);
        }
        else if (author.equals("Any") && !publisher.equals("Any")) {
            stat = conn.prepareStatement(publisherQuery);
            stat.setString(1, publisher);
        }
        else
            stat = conn.prepareStatement(allQuery);

        try (ResultSet rs = stat.executeQuery()) {
            while (rs.next())
                IO.println(rs.getString(1) + ", " + rs.getString(2));
        }
    }

```

```
111     /**
112      * Executes an update statement to change prices.
113      * @param conn the database connection
114      */
115     void changePrices(Connection conn) throws SQLException {
116         String publisher = select("Publishers:", publishers.subList(1, publishers.size()));
117         double priceChange = Double.parseDouble(IO.readln("Change prices by: "));
118         PreparedStatement stat = conn.prepareStatement(priceUpdate);
119         stat.setDouble(1, priceChange);
120         stat.setString(2, publisher);
121         int r = stat.executeUpdate();
122         IO.println(r + " records updated.");
123     }
124
125     /**
126      * Asks the user to select a string.
127      * @param prompt the prompt to display
128      * @param options the options from which the user can choose
129      * @return the option that the user chose
130      */
131     String select(String prompt, List<String> options) {
132         while (true) {
133             IO.println(prompt);
134             for (int i = 0; i < options.size(); i++)
135                 IO.println("%2d) %s".formatted(i + 1, options.get(i)));
136             int sel = Integer.parseInt(IO.readln());
137             if (sel > 0 && sel <= options.size()) return options.get(sel - 1);
138         }
139     }
140 }
```

*java.sql.Connection* **1.1**

- `PreparedStatement prepareStatement(String sql)`
  returns a `PreparedStatement` object containing the precompiled statement. The string `sql` contains a SQL statement with one or more parameter placeholders denoted by `?` characters.

*java.sql.PreparedStatement* **1.1**

- `void setXxx(int n, Xxx x)`
  (Xxx is a type such as `int`, `double`, `String`, `Date`, etc.)
  sets the value of the `n`th parameter to `x`.
- `void clearParameters()`
  clears all current parameters in the prepared statement.
- `ResultSet executeQuery()`
  executes a prepared SQL query and returns a `ResultSet` object.
- `int executeUpdate()`
  executes the prepared SQL `INSERT`, `UPDATE`, or `DELETE` statement represented by the `PreparedStatement` object. Returns the number of rows affected, or `0` for DDL statements such as `CREATE TABLE`.

### 5.5.2. Reading and Writing LOBs

In addition to numbers, strings, and dates, many databases can store *large objects* (LOBs) such as images or other data. In SQL, binary large objects are called BLOBs, and character large objects are called CLOBs.



**Note:** It depends on the application whether it is better to store large objects in the database or a file system. This section tells you what to do once you have decided to use a database for that purpose.

To read a LOB, execute a `SELECT` statement and call the `getBlob` or `getClob` method on the `ResultSet`. You will get an object of type `Blob` or `Clob`. To get the binary data from a `Blob`, call the `getBytes` or `getBinaryStream`. For example, if you have a table with book cover images, you can retrieve an image like this:

```
PreparedStatement pstat
    = conn.prepareStatement("SELECT Cover FROM BookCovers WHERE ISBN=?");
. . .
pstat.set(1, isbn);
try (ResultSet result = pstat.executeQuery()) {
    if (result.next()) {
       Blob coverBlob = result.getBlob(1);
       Image coverImage = ImageIO.read(coverBlob.getBinaryStream());
    }
}
```

Similarly, if you retrieve a `Clob` object, you can get character data by calling the `getSubString` or `getCharacterStream` method.

To place a LOB into a database, call `createBlob` or `createClob` on your `Connection` object, get an output stream or writer to the LOB, write the data, and store the object in the database. For example, here is how you store an image:

```
Blob coverBlob = connection.createBlob();
int offset = 0;
OutputStream out = coverBlob.setBinaryStream(offset);
ImageIO.write(coverImage, "PNG", out);
PreparedStatement pstat = conn.prepareStatement("INSERT INTO Cover VALUES (?, ?)");
pstat.set(1, isbn);
pstat.set(2, coverBlob);
pstat.executeUpdate();
```

*java.sql.ResultSet* **1.1**

- `Blob getBlob(int columnIndex)` **1.2**
- `Blob getBlob(String columnLabel)` **1.2**
- `Clob getClob(int columnIndex)` **1.2**
- `Clob getClob(String columnLabel)` **1.2**

  get the BLOB or CLOB at the given column.

*java.sql.Blob* 1.2

- `long length()`
  gets the length of this BLOB.
- `byte[] getBytes(long startPosition, long length)`
  gets the data in the given range from this BLOB.
- `InputStream getBinaryStream()`
- `InputStream getBinaryStream(long startPosition, long length)`
  return a stream to read the data from this BLOB or from the given range.
- `OutputStream setBinaryStream(long startPosition)` 1.4
  returns an output stream for writing into this BLOB, starting at the given position.

*java.sql.Clob* 1.4

- `long length()`
  gets the number of characters of this CLOB.
- `String getSubString(long startPosition, long length)`
  gets the characters in the given range from this CLOB.
- `Reader getCharacterStream()`
- `Reader getCharacterStream(long startPosition, long length)`
  return a reader (not a stream) to read the characters from this CLOB or from the given range.
- `Writer setCharacterStream(long startPosition)` 1.4
  returns a writer (not a stream) for writing into this CLOB, starting at the given position.

*java.sql.Connection* 1.1

- `Blob createBlob()` 6
- `Clob createClob()` 6
  create an empty BLOB or CLOB.

### 5.5.3. SQL Escapes

The “escape” syntax features are commonly supported by databases but use database-specific syntax variations. It is the job of the JDBC driver to translate the escape syntax to the syntax of a particular database.

Escapes are provided for the following features:

- Date and time literals
- Calling scalar functions
- Calling stored procedures
- Outer joins
- The escape character in `LIKE` clauses

Date and time literals vary widely among databases. To embed a date or time literal, specify the value in one of the ISO 8601/RFC 3339 formats shown below, where the milliseconds are optional. The driver will then translate it into the native format. Use `d`, `t`, `ts` for `DATE`, `TIME`, or `TIMESTAMP` values:

```
{d '2024-01-24'}
{t '23:59:59'}
{ts '2024-01-24 23:59:59.999'}
```

A *scalar function* is a function that returns a single value. Many functions are widely available in databases, but with varying names. The JDBC specification provides standard names and translates them into the database-specific names. To call a function, embed the standard function name and arguments like this:

```
{fn left(?, 20)}
{fn user()}
```

You can find a complete list of supported function names in the JDBC specification.

An *outer join* of two tables does not require that the rows of each table match according to the join condition. For example, the query

```
SELECT * FROM {oj Books LEFT OUTER JOIN Publishers
ON Books.Publisher_Id = Publisher.Publisher_Id}
```

contains books for which `Publisher_Id` has no match in the `Publishers` table, with `NULL` values to indicate that no match exists. You would need a `RIGHT OUTER JOIN` to include publishers without matching books, or a `FULL OUTER JOIN` to return both. Most databases support all forms of outer joins, but some do not. The `oj` escape allows a JDBC driver to locate and rewrite any unsupported outer join operations. If your databases support all forms of outer joins, you need not worry about this escape.

Finally, the `_` and `%` characters have special meanings in a `LIKE` clause—to match a single character or a sequence of characters. There is no standard way to use them literally. If you want to match all strings containing a `_`, use this construct:

```
. . . WHERE ? LIKE %!_% {escape '!'}
```

Here we define `!` as the escape character. The combination `!_` denotes a literal underscore.

A *stored procedure* is a procedure that executes in the database, written in a database-specific language. To invoke a stored procedure, use the `call` escape. You need not supply parentheses if the procedure has no parameters. Use `=` to capture a return value:

```
{call PROC1(?, ?)}
{call PROC2}
{call ? = PROC3(?)}
```

You need to use the `CallableStatement` interface to execute a stored procedure. Set all input parameters and specify the types of any outputs.

```
CallableStatement cstat = conn.prepareCall("{call PROC4(?, ?)}")
cstat.setInt(1, id);
cstat.registerOutParameter(2, java.sql.Types.VARCHAR);
```

```
cstat.execute();
String name = cstat.getString(2);
```

### 5.5.4. Multiple Results

It is possible for a query to return multiple result sets or update counts. This can happen when executing a stored procedure, or with databases that also allow submission of multiple `SELECT` statements in a single query. Here is how you retrieve all result sets:

1. Use the `execute` method to execute the SQL statement.
2. Retrieve the first result set or update count.
3. Repeatedly call the `getMoreResults` method to move on to the next result set or update count.
4. Finish when there are no more result sets or update counts.

The `execute` and `getMoreResults` methods return `true` if the next item in the chain is a result set. The `getUpdateCount` method returns `-1` if the next item in the chain is not an update count.

The following loop traverses all results:

```
boolean isResult = stat.execute(command);
boolean done = false;
while (!done) {
    if (isResult) {
        ResultSet result = stat.getResultSet();
        do something with result
    }
    else {
        int updateCount = stat.getUpdateCount();
        if (updateCount >= 0)
            do something with updateCount
        else
            done = true;
    }
    if (!done) isResult = stat.getMoreResults();
}
```

*java.sql.Statement* **1.1**

- `boolean getMoreResults()`
- `boolean getMoreResults(int current)` **6**
  get the next result for this statement. The `current` parameter is one of `CLOSE_CURRENT_RESULT` (default), `KEEP_CURRENT_RESULT`, or `CLOSE_ALL_RESULTS`. Return `true` if the next result exists and is a result set.

### 5.5.5. Retrieving Autogenerated Keys

Most databases support some mechanism for autonumbering rows in a table. Unfortunately, the mechanisms differ widely among vendors. These automatic numbers are often used as primary keys. Although JDBC doesn't offer a vendor-independent solution for generating keys, it does provide an efficient way of retrieving them. When you insert a new row into a table and a key is automatically generated, you can retrieve it with the following code:

```
stat.executeUpdate(insertStatement, Statement.RETURN_GENERATED_KEYS);
ResultSet rs = stat.getGeneratedKeys();
if (rs.next()) {
    int key = rs.getInt(1);
    . . .
}
```

*java.sql.Statement* **1.1**

- `boolean execute(String statement, int autogenerated)` **1.4**
- `int executeUpdate(String statement, int autogenerated)` **1.4**
  execute the given SQL statement, as previously described. If `autogenerated` is set to `Statement.RETURN_GENERATED_KEYS` and the statement is an `INSERT` statement, the first column contains the autogenerated key.

## 5.6. Scrollable and Updatable Result Sets

As you have seen, the `next` method of the `ResultSet` interface iterates over the rows in a result set. That is certainly adequate for a program that needs to analyze the data. However, consider a visual data display that shows a table or query results (such as **Figure 5.4**). You usually want the user to be able to move both forward and backward in the result set. In a *scrollable* result, you can move forward and backward through a result set and even jump to any position.

Furthermore, once users see the contents of a result set displayed, they may be tempted to edit it. In an *updatable* result set, you can programmatically update entries so that the database is automatically updated. We discuss these capabilities in the following sections.

### 5.6.1. Scrollable Result Sets

By default, result sets are not scrollable or updatable. To obtain scrollable result sets from your queries, you must obtain a different `Statement` object with the method

```
Statement stat = conn.createStatement(type, concurrency);
```

For a prepared statement, use the call

```
PreparedStatement pstat = conn.prepareStatement(command, type, concurrency);
```

The possible values of `type` and `concurrency` are listed in **Table 5.6**. You have the following choices:

- Do you want the result set to be scrollable? If not, use `ResultSet.TYPE_FORWARD_ONLY`.
- If the result set is scrollable, do you want it to reflect changes in the database that occurred after the query that yielded it? (In our discussion, we assume the `ResultSet.TYPE_SCROLL_INSENSITIVE` setting for scrollable result sets. This assumes that the result set does not “sense” database changes that occurred after the execution of the query.)
- Do you want to be able to update the database by editing the result set? (See the next section for details.)

**Table 5.6:** `ResultSet` Type and Concurrency Values

| Value | Explanation |
| --- | --- |
| `TYPE_FORWARD_ONLY` | The result set is not scrollable (default). |
| `TYPE_SCROLL_INSENSITIVE` | The result set is scrollable but not sensitive to database changes. |
| `TYPE_SCROLL_SENSITIVE` | The result set is scrollable and sensitive to database changes. |
| `CONCUR_READ_ONLY` | The result set cannot be used to update the database (default). |
| `CONCUR_UPDATABLE` | The result set can be used to update the database. |

For example, if you simply want to be able to scroll through a result set but don't want to edit its data, use

```
Statement stat = conn.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
```

All result sets that are returned by calls

```
ResultSet rs = stat.executeQuery(query);
```

are now scrollable. A scrollable result set has a *cursor* that indicates the current position.

---



**Note:** Not all database drivers support scrollable or updatable result sets. (The `supportsResultSetType` and `supportsResultSetConcurrency` methods of the `DatabaseMetaData` interface will tell you which types and concurrency modes are supported by a particular database using a particular driver.) Even if a database supports all result set modes, a particular query might not be able to yield a result set with all the properties that you requested. (For example, the result set of a complex query might not be updatable.) In that case, the `executeQuery` method returns a `ResultSet` of lesser capabilities and adds a `SQLWarning` to the connection object. (**Section 5.4.3** shows how to retrieve the warning.) Alternatively, you can use the `getType` and `getConcurrency` methods of the `ResultSet` interface to find out what mode a result set actually has. If you do not check the result set capabilities and issue an unsupported operation, such as `previous` on a result set that is not scrollable, the operation will throw a `SQLException`.

---

Scrolling is very simple. Use

```
if (rs.previous()) . . .;
```

to scroll backward. The method returns `true` if the cursor is positioned on an actual row, or `false` if it is now positioned before the first row.

You can move the cursor backward or forward by any number of rows with the call

```
rs.relative(n);
```

If `n` is positive, the cursor moves forward. If `n` is negative, it moves backward. If `n` is zero, the call has no effect. If you attempt to move the cursor outside the current set of rows, it is set to point either after the last row or before the first row, depending on the sign of `n`. Then, the method returns `false` and the cursor does not move. The method returns `true` if the cursor is positioned on an actual row.

Alternatively, you can set the cursor to a particular row number:

```
rs.absolute(n);
```

To get the current row number, call

```
int currentRow = rs.getRow();
```

The first row in the result set has number `1`. If the return value is `0`, the cursor is not currently on a row—it is either before the first row or after the last row.

The convenience methods `first`, `last`, `beforeFirst`, and `afterLast` move the cursor to the first, to the last, before the first, or after the last position.

Finally, the methods `isFirst`, `isLast`, `isBeforeFirst`, and `isAfterLast` test whether the cursor is at one of these special positions.

Using a scrollable result set is very simple. The hard work of caching the query data is carried out behind the scenes by the database driver.

### 5.6.2. Updatable Result Sets

If you want to edit the data in the result set and have the changes automatically reflected in the database, create an updatable result set. Updatable result sets don't have to be scrollable, but if you present data to a user for editing, you usually want to allow scrolling as well.

To obtain updatable result sets, create a statement as follows:

```
Statement stat = conn.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);
```

The result sets returned by a call to `executeQuery` are then updatable.



**Note:** Not all queries return updatable result sets. If your query is a join that involves multiple tables, the result might not be updatable. However, if your query involves only a single table or if it joins multiple tables by their primary keys, you should expect the result set to be updatable. Call the `getConcurrency` method of the `ResultSet` interface to find out for sure.

For example, suppose you want to raise the prices of some books, but you don't have a simple criterion for issuing an `UPDATE` statement. Then, you can iterate through all books and update prices based on arbitrary conditions.

```
String query = "SELECT * FROM Books";
ResultSet rs = stat.executeQuery(query);
while (rs.next()) {
    if (. . .) {
        double increase = . . .;
        double price = rs.getDouble("Price");
        rs.updateDouble("Price", price + increase);
        rs.updateRow(); // make sure to call updateRow after updating fields
    }
}
```

There are `updateXxx` methods for all data types that correspond to SQL types, such as `updateDouble`, `updateString`, and so on; specify the name or the number of the column (as with the `getXxx` methods), then the new value for the field.

---



**Note:** If you use the `updateXxx` method whose first parameter is the column number, be aware that this is the column number in the *result set*. It could well be different from the column number in the database.

---

The `updateXxx` method changes only the row values, not the database. When you are done with the field updates in a row, you must call the `updateRow` method. That method sends all updates in the current row to the database. If you move the cursor to another row without calling `updateRow`, this row's updates are discarded from the row set and never communicated to the database. You can also call the `cancelRowUpdates` method to cancel the updates to the current row.

The preceding example shows how to modify an existing row. If you want to add a new row to the database, first use the `moveToInsertRow` method to move the cursor to a special position, called the *insert row*. Then, build up a new row in the insert row position by issuing `updateXxx` instructions. When you are done, call the `insertRow` method to deliver the new row to the database. When you are done inserting, call `moveToCurrentRow` to move the cursor back to the position before the call to `moveToInsertRow`. Here is an example:

```
rs.moveToInsertRow();
rs.updateString("Title", title);
rs.updateString("ISBN", isbn);
rs.updateString("Publisher_Id", pubid);
rs.updateDouble("Price", price);
rs.insertRow();
rs.moveToCurrentRow();
```

Note that you cannot influence *where* the new data is added in the result set or the database.

If you don't specify a column value in the insert row, it is set to a SQL `NULL`. However, if the column has a `NOT NULL` constraint, an exception is thrown and the row is not inserted.

Finally, you can delete the row under the cursor:

```
rs.deleteRow();
```

The `deleteRow` method immediately removes the row from both the result set and the database.

The `updateRow`, `insertRow`, and `deleteRow` methods of the `ResultSet` interface give you the same power as executing `UPDATE`, `INSERT`, and `DELETE` SQL statements. However, Java programmers might find it more natural to manipulate the database contents through result sets than by constructing SQL statements.



**Caution:** If you are not careful, you can write staggeringly inefficient code with updatable result sets. It is *much* more efficient to execute an `UPDATE` statement than to make a query and iterate through the result, changing data along the way. Updatable result sets make sense for interactive programs in which a user can make arbitrary changes, but for most programmatic changes, a SQL `UPDATE` is more appropriate.



**Note:** JDBC 2 delivered further enhancements to result sets, such as the capability to update a result set with the most recent data if the data have been modified by another concurrent database connection. JDBC 3 added yet another refinement, specifying the behavior of result sets when a transaction is committed. However, these advanced features are outside the scope of this introductory chapter. We refer you to the *JDBC™ API Tutorial and Reference, Third Edition*, by Maydene Fisher, Jon Ellis, and Jonathan Bruce (Addison-Wesley, 2003) and the JDBC specification for more information.

*java.sql.Connection* **1.1**

- `Statement createStatement(int type, int concurrency)` **1.2**
- `PreparedStatement prepareStatement(String command, int type, int concurrency)` **1.2**
  create a statement or prepared statement that yields result sets with the given type and concurrency. The `type` parameter is of the constants `TYPE_FORWARD_ONLY`, `TYPE_SCROLL_INSENSITIVE`, or `TYPE_SCROLL_SENSITIVE`, and `concurrency` is one of the constants `CONCUR_READ_ONLY` or `CONCUR_UPDATABLE`, all defined in the `ResultSet` interface.

*java.sql.ResultSet* **1.1**

- `int getType()` **1.2**
  returns the type of this result set—one of `TYPE_FORWARD_ONLY`, `TYPE_SCROLL_INSENSITIVE`, or `TYPE_SCROLL_SENSITIVE`.
- `int getConcurrency()` **1.2**
  returns the concurrency setting of this result set—one of `CONCUR_READ_ONLY` or `CONCUR_UPDATABLE`.
- `boolean previous()` **1.2**
  moves the cursor to the preceding row. Returns `true` if the cursor is positioned on a row, or `false` if the cursor is positioned before the first row.
- `int getRow()` **1.2**
  gets the number of the current row. Rows are numbered starting with `1`.
- `boolean absolute(int r)` **1.2**
  moves the cursor to row `r`. Returns `true` if the cursor is positioned on a row.
- `boolean relative(int d)` **1.2**
  moves the cursor by `d` rows. If `d` is negative, the cursor is moved backward. Returns `true` if the cursor is positioned on a row.
- `boolean first()` **1.2**
- `boolean last()` **1.2**
  move the cursor to the first or last row. Return `true` if the cursor is positioned on a row.

- `void beforeFirst()` 1.2
- `void afterLast()` 1.2
  move the cursor before the first or after the last row.
- `boolean isFirst()` 1.2
- `boolean isLast()` 1.2
  test whether the cursor is at the first or last row.
- `boolean isBeforeFirst()` 1.2
- `boolean isAfterLast()` 1.2
  test whether the cursor is before the first or after the last row.
- `void moveToInsertRow()` 1.2
  moves the cursor to the insert row. The insert row is a special row for inserting new data with the `updateXxx` and `insertRow` methods.
- `void moveToCurrentRow()` 1.2
  moves the cursor back from the insert row to the row that it occupied when the `moveToInsertRow` method was called.
- `void insertRow()` 1.2
  inserts the contents of the insert row into the database and the result set.
- `void deleteRow()` 1.2
  deletes the current row from the database and the result set.
- `void updateXxx(int column, Xxx data)` 1.2
- `void updateXxx(String columnName, Xxx data)` 1.2
  (Xxx is a type such as `int`, `double`, `String`, `Date`, etc.)
  update a field in the current row of the result set.
- `void updateRow()` 1.2
  sends the current row updates to the database.
- `void cancelRowUpdates()` 1.2
  cancels the current row updates.

*java.sql.DatabaseMetaData* 1.1

- `boolean supportsResultSetType(int type)` 1.2
  returns `true` if the database can support result sets of the given type; `type` is one of the constants `TYPE_FORWARD_ONLY`, `TYPE_SCROLL_INSENSITIVE`, or `TYPE_SCROLL_SENSITIVE` of the `ResultSet` interface.
- `boolean supportsResultSetConcurrency(int type, int concurrency)` 1.2
  returns `true` if the database can support result sets of the given combination of type and concurrency. The `type` parameter is one of the constants `TYPE_FORWARD_ONLY`, `TYPE_SCROLL_INSENSITIVE`, or `TYPE_SCROLL_SENSITIVE`, and `concurrency` is one of the constants `CONCUR_READ_ONLY` or `CONCUR_UPDATABLE`, all defined in the `ResultSet` interface.

## 5.7. Row Sets

Scrollable result sets are powerful, but they have a major drawback. You need to keep the database connection open during the entire user interaction. However, a user can walk away from the computer for a long time, leaving the connection occupied. That is not good—database connections are scarce resources. In this situation, use a *row set*. The `RowSet` interface extends the `ResultSet` interface, but row sets don't have to be tied to a database connection.

Row sets are also suitable if you need to move a query result to a different tier of a complex application, or to another device such as a cell phone. You would never want to move a result set—its data structures can be huge, and it is tethered to the database connection.

### 5.7.1. Constructing Row Sets

The `javax.sql.rowset` package provides the following interfaces that extend the `RowSet` interface:

- A `CachedRowSet` allows disconnected operation. We will discuss cached row sets in the following section.
- A `WebRowSet` is a cached row set that can be saved to an XML file. The XML file can be moved to another tier of a web application where it is opened by another `WebRowSet` object.
- The `FilteredRowSet` and `JoinRowSet` interfaces support lightweight operations on row sets that are equivalent to SQL `SELECT` and `JOIN` operations. These operations are carried out on the data stored in row sets, without having to make a database connection.
- A `JdbcRowSet` is a thin wrapper around a `ResultSet`. It adds useful methods from the `RowSet` interface.

To obtain a cached row set, call:

```
RowSetFactory factory = RowSetProvider.newFactory();
CachedRowSet crs = factory.createCachedRowSet();
```

There are similar methods for obtaining the other row set types.

### 5.7.2. Cached Row Sets

A cached row set contains all data from a result set. Since `CachedRowSet` is a subinterface of the `ResultSet` interface, you can use a cached row set exactly as you would use a result set. Cached row sets confer an important benefit: You can close the connection and still use the row set. As you will see in our sample program in **Listing 5.4**, this greatly simplifies the implementation of interactive applications. Each user command simply opens the database connection, issues a query, puts the result in a cached row set, and then closes the database connection.

It is even possible to modify the data in a cached row set. Of course, the modifications are not immediately reflected in the database; you need to make an explicit request to accept the accumulated changes. The `CachedRowSet` then reconnects to the database and issues SQL statements to write the accumulated changes.

You can populate a `CachedRowSet` from a result set:

```
ResultSet result = . . .;
RowSetFactory factory = RowSetProvider.newFactory();
CachedRowSet crs = factory.createCachedRowSet();
crs.populate(result);
conn.close(); // now OK to close the database connection
```

Alternatively, you can let the `CachedRowSet` object establish a connection automatically. Set up the database parameters:

```
crs.setURL("jdbc:derby://localhost:1527/COREJAVA");
crs.setUsername("dbuser");
```

```
crs.setPassword("secret");
```

Then set the query statement and any parameters:

```
crs.setCommand("SELECT * FROM Books WHERE Publisher_Id = ?");
crs.setString(1, publisherId);
```

Finally, populate the row set with the query result:

```
crs.execute();
```

This call establishes a database connection, issues the query, populates the row set, and disconnects.

If your query result is very large, you would not want to put it into the row set in its entirety. After all, your users will probably only look at a few rows. In that case, specify a page size:

```
CachedRowSet crs = . . .;
crs.setCommand(command);
crs.setPageSize(20);
. . .
crs.execute();
```

Now you will only get 20 rows. To get the next batch of rows, call

```
crs.nextPage();
```

You can inspect and modify the row set with the same methods you use for result sets. If you modified the row set contents, you must write it back to the database by calling

```
crs.acceptChanges(conn);
```

or

```
crs.acceptChanges();
```

The second call works only if you configured the row set with the information required to connect to a database (such as the URL, username, and password).

In **Section 5.6.2**, you saw that not all result sets are updatable. Similarly, a row set that contains the result of a complex query will not be able to write its changes back to the database. You should be safe if your row set contains data from a single table.

---



**Caution:** If you populated the row set from a result set, the row set does not know the name of the table to update. You need to call `setTableName` to set the table name.

---

Another complexity arises if the data in the database have changed after you populated the row set. This is clearly a sign of trouble that could lead to inconsistent data. The reference implementation checks whether the original row set values (that is, the values before editing) are identical to the current values in the database. If so, they are replaced with the edited values; otherwise, a `SyncProviderException` is thrown and none of the changes are written. Other implementations may use other strategies for synchronization.

*javax.sql.RowSet* **1.4**

- `String getURL()`
- `void setURL(String url)`
  get or set the database URL.
- `String getUsername()`
- `void setUsername(String username)`
  get or set the username for connecting to the database.
- `String getPassword()`
- `void setPassword(String password)`
  get or set the password for connecting to the database.
- `String getCommand()`
- `void setCommand(String command)`
  get or set the command that is executed to populate this row set.
- `void execute()`
  populates this row set by issuing the statement set with `setCommand`. For the driver manager to obtain a connection, the URL, username, and password must be set.

*javax.sql.rowset.CachedRowSet* **5.0**

- `void execute(Connection conn)`
  populates this row set by issuing the statement set with `setCommand`. This method uses the given connection *and closes it*.
- `void populate(ResultSet result)`
  populates this cached row set with the data from the given result set.
- `String getTableName()`
- `void setTableName(String tableName)`
  get or set the name of the table from which this cached row set was populated.
- `int getPageSize()`
- `void setPageSize(int size)`
  get or set the page size.
- `boolean nextPage()`
- `boolean previousPage()`
  load the next or previous page of rows. Return `true` if there is a next or previous page.
- `void acceptChanges()`
- `void acceptChanges(Connection conn)`
  reconnect to the database and write the changes that are the result of editing the row set. May throw a `SyncProviderException` if the data cannot be written back because the database data have changed.

`javax.sql.rowset.RowSetProvider` 7

- `static RowSetFactory newFactory()`
  creates a row set factory.

*javax.sql.rowset.RowSetFactory* 7

- `CachedRowSet createCachedRowSet()`
- `FilteredRowSet createFilteredRowSet()`
- `JdbcRowSet createJdbcRowSet()`
- `JoinRowSet createJoinRowSet()`
- `WebRowSet createWebRowSet()`
  create a row set of the specified type.

## 5.8. Metadata

In the preceding sections, you saw how to populate, query, and update database tables. However, JDBC can give you additional information about the *structure* of a database and its tables. For example, you can get a list of the tables in a particular database or the column names and types of a table. This information is not useful when you are implementing a business application with a predefined database. After all, if you design the tables, you know their structure. Structural information is, however, extremely useful for programmers who write tools that work with any database.

In SQL, data that describe the database or one of its parts are called *metadata* (to distinguish them from the actual data stored in the database). You can get three kinds of metadata: about a database, about a result set, and about parameters of prepared statements.

To find out more about the database, request an object of type `DatabaseMetaData` from the database connection.

```
DatabaseMetaData meta = conn.getMetaData();
```

Use this object to get information about the database. For example, the call

```
ResultSet mrs = meta.getTables(null, null, null, new String[] { "TABLE" });
```

returns a result set that contains information about all tables in the database. (See the API note at the end of this section for other parameters to this method.)

Each row in the result set contains information about a table in the database. The third column is the name of the table. (Again, see the API note for the other columns.) The following loop gathers all table names:

```
while (mrs.next())
    tableNames.add(mrs.getString(3));
```

There is a second important use for database metadata. Databases are complex, and the SQL standard leaves plenty of room for variability. Well over a hundred methods in the `DatabaseMetaData` interface can inquire

about the database, including calls with such exotic names as

```
meta.supportsCatalogsInPrivilegeDefinitions()
```

and

```
meta.nullPlusNonNullIsNull()
```

Clearly, these are geared toward advanced users with special needs—in particular, those who need to write highly portable code that works with multiple databases.

The `DatabaseMetaData` interface gives data about the database. A second metadata interface, `ResultSetMetaData`, reports information about a result set. Whenever you have a result set from a query, you can inquire about the number of columns and each column's name, type, and field width. Here is a typical loop:

```
ResultSet rs = stat.executeQuery("SELECT * FROM " + tableName);
ResultSetMetaData rsmd = rs.getMetaData();
for (int i = 1; i <= rsmd.getColumnCount(); i++) {
    String columnName = rsmd.getColumnLabel(i);
    int columnWidth = rsmd.getColumnDisplaySize(i);
    . . .
}
```

If you have a prepared statement, you can get the metadata of the result set that is returned when the statement is executed:

```
PreparedStatement pstat = conn.prepareStatement(. . .);
ResultSetMetaData rsmd = pstat.getMetaData();
```

A third kind of metadata describes the parameters of a prepared statement:

```
ParameterMetaData pmd = pstat.getParameterMetaData();
for (int i = 1; i <= pmd.getParameterCount()) {
    IO.println(pmd.getParameterTypeName(i));
}
```

The program in **Listing 5.4** uses metadata to let you browse all tables in a database and carry out simple edits. The program also illustrates the use of a cached row set.

The companion code has a prettier GUI version. The combo box on top displays all tables in the database. Select one of them, and the center of the frame is filled with the field names of that table and the values of the first row, as shown in **Figure 5.6**. Click Next and Previous to scroll through the rows in the table. You can also delete a row and edit values of type `String`. Click the Save button to save the changes to the database.

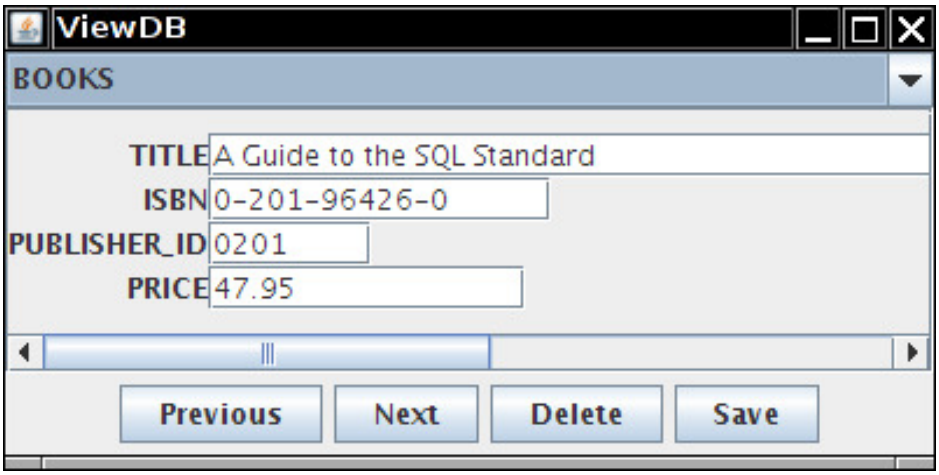



**Figure 5.6:** The `ViewDB` GUI application

**Note:** The example program shows you how to implement tools for working with arbitrary tables. Many databases and IDEs come with much more sophisticated tools for viewing and editing tables. If you prefer a standalone program, check out DBeaver (**https://dbeaver.io**) or SQuirreL (**https://squirrelsql.org**). These programs can be used with any JDBC database.

**Listing 5.4 v2ch05/view/ViewDB2.java**

```
1  package v2ch05.view;
2
3  import module java.base;
4  import module java.sql;
5  import module java.sql.rowset;
6  import v2ch05.exec.ExecSQL;
7  import v2ch07.util.Choices;
8
9  /**
10  * This program uses metadata to display arbitrary tables in a database.
11  */
12 class ViewDB2 {
13     CachedRowSet crs;
14
15     record Column(String label, int width, String columnClass) {
16     }
17
18     List<Column> columns = new ArrayList<>();
19
20     void main() {
21         try (Connection conn = ExecSQL.getConnection()) {
22             DatabaseMetaData meta = conn.getMetaData();
23             var tableNames = new ArrayList<String>();
24
25             try (ResultSet mrs = meta.getTables(null, null, null, new String[]{ "TABLE" })) {
26                 while (mrs.next())
27                     tableNames.add(mrs.getString(3));
28             }
29             IO.println("Choose a table");
30             String selected = Choices.choose(tableNames);
31             showTable(selected, conn);
32             executeCommands();
33             crs.acceptChanges(conn);
34         }
```

```
        catch (SQLException e) {
            for (Throwable t : e)
                t.printStackTrace();
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * Prepares the columns list for showing a new table, and shows the first row.
     * @param tableName the name of the table to display
     * @param conn the database connection
     */
    void showTable(String tableName, Connection conn) throws SQLException {
        try (Statement stat = conn.createStatement();
             ResultSet result = stat.executeQuery("SELECT * FROM " + tableName)) {
            // copy into cached row set
            RowSetFactory factory = RowSetProvider.newFactory();
            crs = factory.createCachedRowSet();
            crs.setTableName(tableName);
            crs.populate(result);

            ResultSetMetaData rsmd = result.getMetaData();
            for (int i = 1; i <= rsmd.getColumnCount(); i++) {
                columns.add(new Column(rsmd.getColumnLabel(i), rsmd.getColumnDisplaySize(i),
                        rsmd.getColumnClassName(i)));
            }

            showNextRow();
        }
    }

    void executeCommands() throws SQLException {
        while (true) {
            String selection = Choices.choose("Next", "Previous", "Edit", "Delete", "Quit");
            switch (selection) {
                case "Next" -> showNextRow();
                case "Previous" -> showPreviousRow();
                case "Edit" -> editRow();
                case "Delete" -> deleteRow();
                default -> {
                    return;
                }
            }
        }
    }

    /**
     * Shows a database row by populating all text fields with the column values.
     */
    void showRow(ResultSet rs) throws SQLException {
        for (int i = 1; i <= columns.size(); i++) {
            String format = "%%s [%%%ds]".formatted(columns.get(i - 1).width());
            IO.println(format.formatted(columns.get(i - 1).label(),
                    rs == null ? "" : rs.getString(i)));
        }
```

```
92      }
93
94      /**
95       * Moves to the previous table row.
96       */
97      void showPreviousRow() throws SQLException {
98          if (crs == null || crs.isFirst()) return;
99          crs.previous();
100         showRow(crs);
101     }
102
103     /**
104      * Moves to the next table row.
105      */
106     void showNextRow() throws SQLException {
107         if (crs == null || crs.isLast()) return;
108         crs.next();
109         showRow(crs);
110     }
111
112     /**
113      * Deletes current table row.
114      */
115     void deleteRow() throws SQLException {
116         if (crs == null) return;
117         crs.deleteRow();
118         if (crs.isAfterLast()) {
119             if (!crs.last()) crs = null;
120         }
121         showRow(crs);
122     }
123
124     /**
125      * Updates changed data into the current row of the row set.
126      */
127     void editRow() throws SQLException {
128         List<String> editableColumns = columns.stream()
129                 .filter(c -> c.columnClass.equals("java.lang.String")).map(Column::label)
130                 .toList();
131         IO.println("Choose column");
132         String label = Choices.choose(editableColumns);
133         String newValue = IO.readln("New value: ");
134         crs.updateString(label, newValue);
135         crs.updateRow();
136     }
137 }
```

*java.sql.Connection* **1.1**

- `DatabaseMetaData getMetaData()`
  returns the metadata for the connection as a `DatabaseMetaData` object.

*java.sql.DatabaseMetaData* **1.1**

- `ResultSet getTables(String catalog, String schemaPattern, String tableNamePattern, String types[])`
  returns a description of all tables in a catalog that match the schema and table name patterns and the type criteria. (A *schema* describes a group of related tables and access permissions. A *catalog* describes a related group of schemas. These concepts are important for structuring large databases.)
  The `catalog` and `schemaPattern` parameters can be `""` to retrieve those tables without a catalog or schema, or `null` to return tables regardless of catalog or schema.
  The `types` array contains the names of the table types to include. Typical types are `TABLE`, `VIEW`, `SYSTEM TABLE`, `GLOBAL TEMPORARY`, `LOCAL TEMPORARY`, `ALIAS`, and `SYNONYM`. If `types` is `null`, tables of all types are returned.
  The result set has five columns, all of which are of type `String`.

| Column | Name | Explanation |
|---|---|---|
| `1` | `TABLE_CAT` | Table catalog (may be `null`) |
| `2` | `TABLE_SCHEM` | Table schema (may be `null`) |
| `3` | `TABLE_NAME` | Table name |
| `4` | `TABLE_TYPE` | Table type |
| `5` | `REMARKS` | Comment on the table |

- `int getJDBCMajorVersion()` **1.4**
- `int getJDBCMinorVersion()` **1.4**
  return the major or minor JDBC version numbers of the driver that established the database connection. For example, a JDBC 4.3 driver has major version number 4 and minor version number 3.
- `int getMaxConnections()`
  returns the maximum number of concurrent connections allowed to this database.
- `int getMaxStatements()`
  returns the maximum number of concurrently open statements allowed per database connection, or `0` if the number is unlimited or unknown.

*java.sql.ResultSet* **1.1**

- `ResultSetMetaData getMetaData()`
  returns the metadata associated with the current `ResultSet` columns.

*java.sql.PreparedStatement* **1.1**

- `ResultSetMetaData getMetaData()` **1.2**
  returns the metadata associated with the result set that is returned when this statement is executed.
- `ParameterMetaData getMetaData()` **1.4**
  returns metadata describing the parameters of this statement.

*java.sql.ResultSetMetaData* **1.1**

- `int getColumnCount()`
  returns the number of columns in the current `ResultSet` object.
- `String getColumnName(int column)`
- `String getColumnLabel(int column)`
- `int getColumnDisplaySize(int column)`
  return the name, suggested title, and display size of the column with the given one-based index.

*java.sql.ParameterMetaData* **1.4**

- `int getParameterCount()`
  returns the number of parameters of the prepared statement.
- `int getParameterType(int param)`
- `String getParameterTypeName(int param)`
- `String getParameterClassName(int param)`
  return the type of the parameter with the given one-based index, as a constant defined in `java.sql.Types`, or as a database-specific type name, or as a fully-qualified Java class name.
- `int getParameterMode(int param)`
  returns the mode of the parameter with the given one-based index, as one of the static constants `parameterModeIn`, `parameterModeOut`, `parameterModeInOut`, or `parameterModeUnknown`.
- `int isNullable(int param)`
  returns the nullability mode of the parameter with the given one-based index, as one of the static constants `parameterNoNulls`, `parameterNullable`, or `parameterNullableUnknown`.
- `int getPrecision(int param)`
  returns the maximum column size of the parameter with the given one-based index. For non-numeric data types, this is the length of the string or string representation, or the number of bytes for binary data.
- `int getScale(int param)`
  returns the number of digits after the decimal point of the parameter with the given one-based index, or 0 for non-numeric data.

## 5.9. Transactions

You can group a set of statements to form a *transaction*. The transaction can be *committed* when all has gone well—or, if an error has occurred in one of them, it can be *rolled back* as if none of the statements had been issued.

The major reason for grouping statements into transactions is *database integrity*. For example, suppose we want to transfer money from one bank account to another. Then, it is important that we simultaneously debit one account and credit another. If the system fails after debiting the first account but before crediting the other account, the debit needs to be undone.

If you group update statements into a transaction, the transaction either succeeds in its entirety and can be *committed,* or it fails somewhere in the middle. In that case, you can carry out a *rollback* and the database automatically undoes the effect of all updates that occurred since the last committed transaction.

### 5.9.1. Programming Transactions with JDBC

By default, a database connection is in *autocommit mode*, and each SQL statement is committed to the database as soon as it is executed. Once a statement is committed, you cannot roll it back. Turn off this default so you can use transactions:

```
conn.setAutoCommit(false);
```

Create a statement object in the normal way:

```
Statement stat = conn.createStatement();
```

Call `executeUpdate` any number of times:

```
stat.executeUpdate(command1);
stat.executeUpdate(command2);
stat.executeUpdate(command3);
. . .
```

If all statements have been executed without error, call the `commit` method:

```
conn.commit();
```

However, if an error occurred, call

```
conn.rollback();
```

Then, all statements since the last commit are automatically reversed. You typically issue a rollback when your transaction was interrupted by a `SQLException`.

### 5.9.2. Save Points

With some databases and drivers, you can gain finer-grained control over the rollback process by using *save points*. Creating a save point marks a point to which you can later return without having to abandon the entire transaction. For example,

```
Statement stat = conn.createStatement(); // start transaction; rollback() goes here
stat.executeUpdate(command1);
Savepoint svpt = conn.setSavepoint(); // set savepoint; rollback(svpt) goes here
stat.executeUpdate(command2);

if (. . .) conn.rollback(svpt); // undo effect of command2
. . .
conn.commit();
```

When you no longer need a save point, you should release it:

```
conn.releaseSavepoint(svpt);
```

### 5.9.3. Batch Updates

Suppose a program needs to execute many `INSERT` statements to populate a database table. You can improve the performance of the program by using a *batch update*. In a batch update, a sequence of statements is collected and submitted as a batch.

---



**Note:** Use the `supportsBatchUpdates` method of the `DatabaseMetaData` interface to find out if your database supports this feature.

---

The statements in a batch can be actions such as `INSERT`, `UPDATE`, or `DELETE` as well as data definition statements such as `CREATE TABLE` or `DROP TABLE`. An exception is thrown if you add a `SELECT` statement to a batch. (Conceptually, a `SELECT` statement makes no sense in a batch because it returns a result set without updating the database.)

To execute a batch, first create a `Statement` object in the usual way:

```
Statement stat = conn.createStatement();
```

Now, instead of calling `executeUpdate`, call the `addBatch` method:

```
String command = "CREATE TABLE . . ."
stat.addBatch(command);

while (. . .) {
    command = "INSERT INTO . . . VALUES (" + . . . + ")";
    stat.addBatch(command);
}
```

Finally, submit the entire batch:

```
int[] counts = stat.executeBatch();
```

The call to `executeBatch` returns an array of the row counts for all submitted statements.

For proper error handling in batch mode, treat the batch execution as a single transaction. If a batch fails in the middle, you want to roll back to the state before the beginning of the batch.

First, turn the autocommit mode off, then collect the batch, execute it, commit it, and finally restore the original autocommit mode:

```
boolean autoCommit = conn.getAutoCommit();
conn.setAutoCommit(false);
Statement stat = conn.getStatement();
. . .
// keep calling stat.addBatch(. . .);
```

```
. . .
stat.executeBatch();
conn.commit(); conn.setAutoCommit(autoCommit);
```

*java.sql.Connection* **1.1**

- `boolean getAutoCommit()`
- `void setAutoCommit(boolean b)`
  get or set the autocommit mode of this connection to `b`. If autocommit is `true`, all statements are committed as soon as their execution is completed.
- `void commit()`
  commits all statements that were issued since the last commit.
- `void rollback()`
  undoes the effect of all statements that were issued since the last commit.
- `Savepoint setSavepoint()` **1.4**
- `Savepoint setSavepoint(String name)` **1.4**
  set an unnamed or named save point.
- `void rollback(Savepoint svpt)` **1.4**
  rolls back until the given save point.
- `void releaseSavepoint(Savepoint svpt)` **1.4**
  releases the given save point.

*java.sql.Savepoint* **1.4**

- `int getSavepointId()`
  gets the ID of this unnamed save point, or throws a `SQLException` if this is a named save point.
- `String getSavepointName()`
  gets the name of this save point, or throws a `SQLException` if this is an unnamed save point.

*java.sql.Statement* **1.1**

- `void addBatch(String command)` **1.2**
  adds the command to the current batch of commands for this statement.
- `int[] executeBatch()` **1.2**
- `long[] executeLargeBatch()` **8**
  execute all commands in the current batch. Each value in the returned array corresponds to one of the batch statements. If it is non-negative, it is a row count. If it is the value `SUCCESS_NO_INFO`, the statement succeeded, but no row count is available. If it is `EXECUTE_FAILED`, the statement failed.

*java.sql.DatabaseMetaData* **1.1**

- `boolean supportsBatchUpdates()` **1.2**
  returns `true` if the driver supports batch updates.

### 5.9.4. Advanced SQL Types

Table 5.8 lists the SQL data types supported by JDBC and their equivalents in the Java programming language.

**Table 5.8:** SQL Data Types and Their Corresponding Java Types

| SQL Data Type | Java Data Type |
|---|---|
| `INTEGER` or `INT` | `int` |
| `SMALLINT` | `short` |
| `NUMERIC(m,n)`, `DECIMAL(m,n)` or `DEC(m,n)` | `java.math.BigDecimal` |
| `FLOAT(n)` | `double` |
| `REAL` | `float` |
| `DOUBLE` | `double` |
| `CHARACTER(n)` or `CHAR(n)` | `String` |
| `VARCHAR(n)`, `LONG VARCHAR` | `String` |
| `BOOLEAN` | `boolean` |
| `DATE` | `java.sql.Date` |
| `TIME` | `java.sql.Time` |
| `TIMESTAMP` | `java.sql.Timestamp` |
| `BLOB` | `java.sql.Blob` |
| `CLOB` | `java.sql.Clob` |
| `ARRAY` | `java.sql.Array` |
| `ROWID` | `java.sql.RowId` |
| `NCHAR(n)`, `NVARCHAR(n)`, `LONG NVARCHAR` | `String` |
| `NCLOB` | `java.sql.NClob` |
| `SQLXML` | `java.sql.SQLXML` |

A SQL `ARRAY` is a sequence of values. For example, in a `Student` table, you can have a `Scores` column that is an `ARRAY OF INTEGER`. The `getArray` method returns an object of the interface type `java.sql.Array`. That interface has methods to fetch the array values.

When you get a LOB or an array from a database, the actual contents are fetched from the database only when you request individual values. This is a useful performance enhancement, as the data can be quite voluminous.

Some databases support `ROWID` values that describe the location of a row so that it can be retrieved very rapidly. JDBC 4 introduced an interface `java.sql.RowId` and the methods to supply the row ID in queries and retrieve it from results.

A *national character string* (`NCHAR` and its variants) stores strings in a local character encoding and sorts them using a local sorting convention. JDBC 4 provided methods for converting between Java `String` objects and national character strings in queries and results.

Some databases can store user-defined structured types. JDBC 3 provides a mechanism for automatically mapping structured SQL types to Java objects.

Some databases provide native storage for XML data. JDBC 4 introduced a `SQLXML` interface that can mediate between the internal XML representation and the DOM `Source/Result` interfaces, as well as binary streams. See the API documentation for the `SQLXML` class for details.

We do not discuss these advanced SQL types any further. You can find more information on these topics in the *JDBC API Tutorial and Reference* and the JDBC specification.

## 5.10. Connection Management in Web and Enterprise Applications

The simplistic database connection setup with a `database.properties` file, as described in the preceding sections, is suitable for small test programs but won't scale for larger applications.

When a JDBC application is deployed in a web or enterprise environment, the management of database connections is integrated with a directory service called JNDI (Java Naming and Directory Interface). The properties of data sources across the enterprise can be stored in a directory. Using a directory allows for centralized management of usernames, passwords, database names, and JDBC URLs.

In such an environment, you can use the following code to establish a database connection:

```
var jndiContext = new InitialContext();
var source = (DataSource) jndiContext.lookup("java:comp/env/jdbc/corejava");
Connection conn = source.getConnection();
```

Note that the `DriverManager` is no longer involved. Instead, the JNDI service locates a *data source.* A data source is an interface that allows for simple JDBC connections as well as more advanced services, such as executing distributed transactions that involve multiple databases. The `DataSource` interface is defined in the `javax.sql` standard extension package.



**Note:** In a Java EE container, you don't even have to program the JNDI lookup. Simply use the `Resource` annotation on a `DataSource` field, and the data source reference will be set when your application is loaded:

```
@Resource(name="jdbc/corejava")
private DataSource source;
```

Of course, the data source needs to be configured somewhere. If you write database programs that execute in a servlet container such as Apache Tomcat or in an application server such as GlassFish, place the database con-

figuration (including the JNDI name, JDBC URL, username, and password) in a configuration file, or set it in an admin GUI.

Management of usernames and logins is just one of the issues that require special attention. Another issue involves the cost of establishing database connections. Our sample database programs used two strategies for obtaining a database connection. The `QueryDB` program in **Listing 5.3** established a single database connection at the start of the program and closed it at the end of the program. The `ViewDB` program in **Listing 5.4** opened a new connection whenever one was needed.

However, neither of these approaches is satisfactory. Database connections are a finite resource. If a user walks away from an application for some time, the connection should not be left open. Conversely, obtaining a connection for each query and closing it afterward is very costly.

The solution is to *pool* connections. This means that database connections are not physically closed but are kept in a queue and reused. Connection pooling is an important service, and the JDBC specification provides hooks for implementors to supply it. However, the JDK itself does not provide any implementation, and database vendors don't usually include one with their JDBC drivers either. Instead, vendors of web containers and application servers supply connection pool implementations.

Using a connection pool is completely transparent to the programmer. Acquire a connection from a source of pooled connections by obtaining a data source and calling `getConnection`. When you are done using the connection, call `close`. That doesn't close the physical connection but tells the pool that you are done using it. The connection pool typically makes an effort to pool prepared statements as well.

You have now learned about the JDBC fundamentals and know enough to implement simple database applications. However, as we mentioned at the beginning of this chapter, databases are complex and quite a few advanced topics are beyond the scope of this introductory chapter. For an overview of advanced JDBC capabilities, refer to the *JDBC API Tutorial and Reference* or the JDBC specification.

In this chapter, you have learned how to work with relational databases in Java. The next chapter covers the date and time library.

Previous chapter
< 4. Networking

Next chapter
6. The Date and Time API >